

Build Your Powerful AI Agents with Copilot

A working reference for deciding *where* an agent should be built before deciding *what* it does.
Read it end to end, or go straight to the section you need.

Track 2 · Microsoft Frontier Transformation Day · Tuesday, 22 September 2026
Indonesia Room, Shangri-La Hotel, Jakarta

Adelia Chindranata

Copilot and Agents Solution Engineer, Microsoft

CONTENTS

1 Microsoft 365 Copilot

- 1.1 The five capabilities
- 1.2 How it works · code interpreter
- 1.3 Copilot and Cowork

2 Agents and Harness

- 2.1 Harnesses, Microsoft view
- 2.2 Agent Builder and skills
- 2.3 Anatomy of an agent

3 Skills — what they are

4 Planning a Document Comparison Agent

- 4.1 High-level design
- 4.2 Choosing your harness

5 Hands On: RFP Checker Agent

- 5.1 A specialised comparison agent
- 5.2 Workflow and determinism split
- 5.3 Agent design — instructions, skills
- 5.4 Try it yourself! — Build it
- 5.5 Expected output

THE EQUATION

M365 Copilot = LLM + Work IQ + In-app Copilot experience + Web IQ

Every agent in this workbook is built *on* this product, not beside it. Before choosing a harness (Section 2) you need to know what the platform already gives you for free — because capability you can get from the product is capability you should not design into an agent.

Reading the equation, one term at a time

The LLM is the reasoning half, hosted in the Microsoft Cloud. On *Copilot Chat (Basic)* this is GPT; on *Microsoft 365 Copilot (Premium)* you additionally get model choice across GPT and Claude. Two properties hold on both tiers: the models are **not trained on your organisational data**, and Copilot **inherits the security, compliance and privacy policies already set in Microsoft 365** — it does not introduce a second permission model.

Work IQ is, in Microsoft's own words, "*a workplace intelligence layer that enables agents to access and reason over organizational data, context, and tools.*" It continuously builds a semantic understanding across Microsoft 365 and external systems, with permission-aware governance built in. It does not replace Microsoft Graph — the two work together, with Graph remaining the grounding path into tenant content. Learn describes Work IQ as combining **chat, context, tools and workspaces**:

WORK IQ Four capabilities, one intelligence layer

01 CHAT

Conversational intelligence

Supports A2A for agent-to-agent collaboration and REST for web applications, so agents can delegate work, maintain continuity and return fully processed, Copilot-quality responses.

02 CONTEXT

No orchestration overhead

Work IQ assembles and grounds context internally across organisational data to provide agent-ready inputs, so agents do not stitch together raw signals or manage retrieval pipelines.

03 TOOLS

Fast, composable actions

The Work IQ MCP collapses hundreds of operations into **10 generic tools** reaching mail, calendar, files, people, chat and ... sites — simple verbs such as **fetch**, **create** and **update**.

04 WORKSPACES

Long-running agent workflows

SharePoint Embedded working storage gives agents a persistent space inside your tenant boundary for intermediate data and outputs — supporting task progression, reuse of results and handoff between agents.

Two of these matter more than they first appear. **Context** is why the same prompt behaves differently in Copilot than in a generic assistant — the grounding is assembled for you rather than stitched together from raw signals. And **Workspaces** is the one most people miss: persistent storage inside the tenant boundary is exactly what a stateless sandbox cannot offer, which is why it resurfaces as a design constraint in Sections 4.2 and 5.

Developers can reach Work IQ directly from custom apps and agents through the **Work IQ API** — A2A, a remote MCP server and a REST API. Note the commercial detail: "*Work IQ API access is independent of Microsoft 365 Copilot licensing and available through usage-based billing.*" For licensed users it is present across all Copilot experiences and agents, but custom and third-party agents are billed on usage.

The in-app Copilot experience is where the intelligence is spent. On Basic, Copilot reaches into Outlook — and Copilot Chat in Outlook now reasons over the whole inbox, calendar and meetings rather than a single thread. On Premium it spans Outlook, Teams, Word, Excel and PowerPoint, adds *Edit with Copilot*, and brings AI-powered Search, Notebooks and Create. This is the "in the flow of work" half: the same reasoning, surfaced where the artefact already lives instead of in a separate chat window.

Web IQ supplies real-time public web grounding. It is the term for the web-search half of the answer, and it is available on *both* tiers — Copilot Chat's baseline offer is secure, web-grounded chat. The practical consequence for design: web grounding is never the differentiator between tiers; **Work IQ is**.

There aren't two separate chats anymore. There's **one chat box with a single Work IQ toggle** — flip it to control what Copilot grounds its answer in.



WORK IQ OFF

Web-grounded Chat

Copilot reasons over public web knowledge and the files you hand it — ideal for research, drafting and general questions. Every prompt starts with you finding, attaching or pasting the material first.

- Open-web knowledge
- Plus any files you attach
- Available to Basic & Premium

SUGGESTED PROMPTS TO TRY

WITHOUT WORK IQ

Review [/Document A](#) and compare it against [/Document B](#).

Suggest a reply to this question [\[paste the question\]](#).

Notice the pattern. Both prompts begin with you supplying the raw material — the documents, the pasted text. Copilot is capable, but it starts from zero every time.



WORK IQ ON

Work IQ-grounded Chat

Copilot also reasons over your work across Microsoft 365 — emails, chats, meetings and files — trimmed to what you already have permission to see. The context is already there before you type.

- Everything web-grounded does
- Your emails, chats, meetings and files
- Permission-trimmed — nothing you can't already open

SUGGESTED PROMPTS TO TRY

WITH WORK IQ

List my meetings today and the key decisions I should make.

Recap my previous engagements with [\[stakeholder\]](#) — what has been decided, and what has not?

The prompt gets shorter, and the question gets bigger. You stop describing the material and start asking about your actual work.

It's one chat with a single Work IQ toggle — not two separate chat experiences.

SOURCES — SECTION 1

- [Work IQ overview — the intelligence layer](#)
- [Microsoft Copilot service description — what is included per plan](#)
- [Enterprise data protection — training exclusion and policy inheritance](#)

The equation in Section 1 describes what Copilot is *made of*. Customers rarely buy a composition — they buy capability. The sharing-session deck frames Microsoft 365 Copilot as **a unified AI platform that takes the latest innovations and tailors them for work**, expressed as five capabilities. Learn these five as a set: each one is a different place an agent design can either lean on the platform or duplicate it.

THE PLATFORM Five capabilities — and what each means when you are designing

01

Work IQ

The intelligence layer that personalises Copilot to you and your organisation.

Grounding you inherit rather than build. If your agent's knowledge requirement is "the user's own mail, files, meetings and chats", you are describing Work IQ — do not rebuild it as a knowledge source.

02

Agentic Work

AI in the flow of work that turns intent into action.

The platform itself now executes, not just drafts. Multi-step execution over time is a product capability (Cowork, Section 1.3) before it is an agent you have to build.

03

Multi-model

The best model for every task and workflow.

Model choice is a platform dial, not a reason to move to a pro-code surface. Needing "a different model" is rarely a valid harness-escalation argument on its own.

04

Agent Platform

An ecosystem of agents with agent-building tools for everyone.

The built-in build surfaces: Agent Builder and Copilot Studio. This capability is what Sections 2–5 of this workbook unpack.

05

Security & Management

Built-in security, management and governance.

Your agent inherits tenant security, compliance and admin controls by default. Governance is a configuration surface, not a component you author.

Capability 2 in detail — what "agentic work" actually covers

Agentic Work is the capability most often under-described, because it spans three quite different things:

- **In-app action** — Copilot doing the next step inside the app you are already in: triaging mail, accepting a meeting, flagging a message from your manager, editing a document in place.
- **Ready-made agents** — Microsoft-built agents in the experience: Researcher, Analyst, Facilitator, Interpreter, plus Copilot Search, Notebooks, Create and Workflows.
- **Delegated multi-step work** — *Copilot Cowork*, where you hand over an outcome rather than a prompt and the work runs for minutes or hours in the background. This is the subject of Section 1.3.

Capability 4 in detail — the agent platform, by who builds

WHO BUILDS WHAT, ON WHICH SURFACE

BUILDER	SURFACE	GROUNDING AVAILABLE
Copilot Chat (Basic) user	M365 Copilot Agent Builder	Website knowledge
M365 Copilot (Premium) user	M365 Copilot Agent Builder	Website knowledge and Work IQ
Maker / IT	Copilot Studio	Full knowledge, tools, flows and triggers; deployable to M365 Copilot and Teams, with different charging for Basic and Premium users

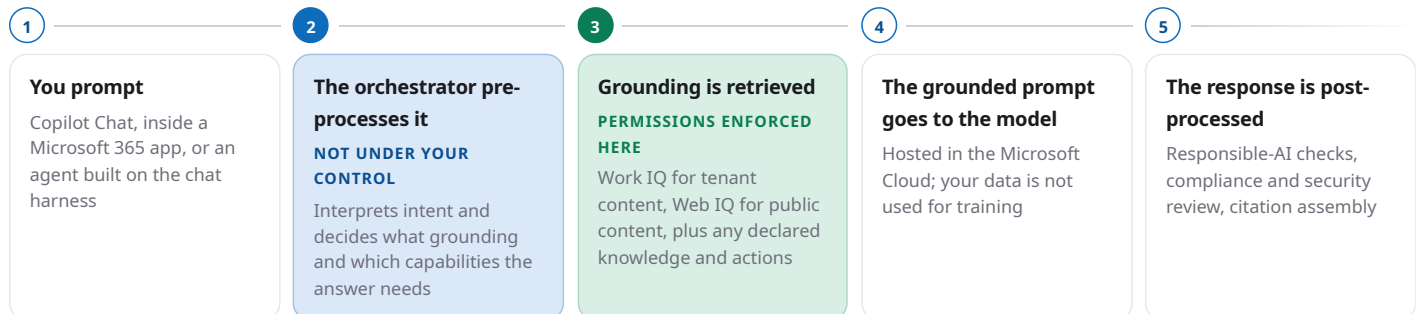
That single row difference — *Work IQ or not* — is the most consequential licensing fact in agent planning. Check which license your intended users hold before you design anything that depends on Work IQ grounding.

SOURCES — SECTION 1.1

- [Build agents using Agent Builder in Microsoft 365 Copilot](#)
- [License options for Microsoft Copilot — what each tier includes](#)
- [Agent Builder in Microsoft 365 Copilot — the built-in build surface](#)

A prompt does not go straight to a model. Understanding the path it does take is what lets you predict which of your agent's requirements the platform will satisfy and which it will quietly drop.

PROMPT TO ANSWER Five stages, and where each one is decided



Grounding and tool calls run sequentially on this harness — not in a loop.

The path a prompt takes

- 1. You prompt** — from Copilot Chat, from inside Word, Outlook, Teams, Excel or PowerPoint, or from an agent built on the chat harness.
- 2. The orchestrator pre-processes it** — it interprets intent and decides what grounding and which capabilities the answer needs. This is the component you do *not* control on the Copilot chat harness (Section 2.3).
- 3. Grounding is retrieved** — Work IQ for tenant content (always within your existing permissions, never more), Web IQ for public content, plus any knowledge sources or actions the agent declares.
- 4. The grounded prompt goes to the model** — hosted in the Microsoft Cloud; your data is not used for training.
- 5. The response is post-processed** — responsible-AI checks, compliance and security review, citation assembly — then returned with the actions the user can take on it.

Two design consequences fall straight out of this. First, **permissions are enforced at grounding time**, so an agent can never widen what a user can see — but it can surface content the user had forgotten they had access to, which is a governance conversation worth having early. Second, on the chat harness, grounding and tool calls run **sequentially rather than in a loop** — so "retrieve, then decide, then retrieve again based on what you found" is not a shape this harness supports.

What code interpreter is

Code interpreter is a capability, not a model. It lets Copilot — and any declarative agent that declares it — *write and run Python in a sandboxed environment managed by Microsoft*, then return both the result and the code it generated. The runtime is ephemeral: it spins up for the computation and tears down afterwards. No Azure subscription, no external service, no API keys.

It exists because language models are unreliable at arithmetic and cannot produce a real file. Code interpreter closes both gaps at once: it computes deterministically, and it emits downloadable artefacts.

CODE INTERPRETER — WHAT IT DOES AND WHERE IT IS CONFIGURED

ASPECT	DETAIL
What it produces	Charts and graphs, QR codes and word clouds, synthetic sample data, solved multi-step maths, modified images, and downloadable Word, Excel, PowerPoint and PDF files — plus a copyable, downloadable version of the Python it ran.
What it accepts as input	Uploaded Word, Excel, PowerPoint, PDF, CSV/TSV and TXT/UTF8 files, for analysis or modification.
Where you enable it — Agent Builder	On by default. Toggle it under <i>Skip to configure</i> → <i>Capabilities</i> → "Create documents, charts, and code".
Where you enable it — Agents Toolkit	<code>{ "capabilities": [{ "name": "CodeInterpreter" }] }</code> in the declarative agent manifest — requires manifest schema v1.2 or later.
Who can use it	Users with a Microsoft 365 Copilot license, and Copilot Chat users without metered usage enabled. Support for in-context agents varies by host.

When does it trigger?

This is the part most builders get wrong. **You do not route to code interpreter — the orchestrator decides.** Declaring the capability tells the orchestrator "this agent is allowed to run code"; it then invokes the sandbox when it judges that a request needs computation, a generated file, or a visualisation it cannot produce in prose. There is no branch to author, no condition to write, and no guarantee it fires on any particular turn.

Because it is probabilistic, the reliable lever is *instructions*. A line such as "*When performing calculations involving rates, accruals or financial projections, use the code interpreter to ensure accuracy*" materially increases the hit rate. That is a design pattern worth adopting whenever numbers appear in an answer users will act on.

The runtime in brief — and where it is documented

Two Microsoft Learn pages carry everything Microsoft publishes about this capability, and it is worth knowing which one answers which question:

- **Add the code interpreter capability to your agent** — the functional brief: what it does, what it accepts, how to enable it, and the file-hosting rule. Last updated 18 June 2026.
- **Code interpreter security architecture** — the runtime brief: isolation, session lifecycle, network posture and resource quotas. Last updated 26 November 2025.

A third page, *Use code interpreter in a prompt*, covers the same capability on the **standard harness** in Copilot Studio — a different surface with different constraints, worth reading only when you have already chosen that harness.

PROPERTY	DOCUMENTED BEHAVIOUR
Where it runs	Isolated virtual machines hosted on Microsoft Azure. Each session is separated from every other session <i>and</i> from core Copilot services, so there is no cross-session contamination.
Lifetime	A fresh VM every time code runs . The VM is destroyed when the session ends, and no data is persisted or stored — the design deliberately prevents persistent storage of code or data.
Network	No inbound or outbound traffic at all . The sandbox cannot reach an external server, which rules out exfiltration — and equally rules out calling an API, fetching a URL, or installing a package at runtime.
Capacity	Resource quotas bound time, CPU, memory and disk , to mitigate denial-of-service risk and keep allocation fair. The quotas exist and are enforced; Microsoft does not publish their numeric values.
Output files	Generated files are hosted temporarily and downloadable during the active session only . They are <i>not</i> persisted after the session ends. Copilot surfaces a download link automatically when a file is produced.
Input files	Word, Excel, PowerPoint, PDF, CSV/TSV and TXT/UTF8, accepted for analysis or modification.
Runtime monitoring	All code is scanned in real time for malicious patterns. Attempts to reach restricted files or escalate privileges terminate the session immediately and alert security teams.
Compliance posture	Runs inside Microsoft 365 data residency boundaries, under policies aligned to ISO 27001, SOC 2 and GDPR. Foundation models are not trained on customer data.

What is preinstalled

Microsoft does not publish a package manifest for the code interpreter environment. There is no documented list of Python libraries and no supported way to query one, and because the sandbox has no outbound network, **you cannot install anything at runtime either**. The environment is fixed, opaque and not extensible.

What the documentation gives you instead is *evidence of capability*, which is the more useful thing to design against. The published examples confirm the environment can, without any setup on your part:

- Write **Word, Excel, PowerPoint and PDF** files, and read those same formats plus CSV/TSV and TXT/UTF8 as input.
- Render **charts and graphs** — line graphs, bar charts — from computed or uploaded data.
- Produce **QR codes and word clouds**.
- Perform **image manipulation** on uploads — greyscale conversion, banners, captions.
- Solve **symbolic and numerical maths**, including definite integrals.

Treat that as the contract. If your scenario needs something outside it — a niche statistical package, a proprietary SDK, an external API call mid-computation — this harness cannot host it, and you are looking at a harness decision rather than a configuration one.

DESIGN CONSEQUENCES OF THE LIFETIME RULE

Because the VM is created per run and destroyed with the session, three things follow that catch teams out. **There is no state between runs** — a variable, dataframe or intermediate file from an earlier computation is gone, so any multi-step analysis must be re-derived from inputs each time. **There is no artefact archive** — if the user does not download the file during the session, it is not recoverable, so an agent whose value depends on a durable deliverable needs somewhere real to write it. And **long-running work has no home here** — the quotas bound execution time deliberately. Cowork's persistence model (Section 1.3) exists precisely because these three limits are real.

SOURCES — SECTION 1.2

- [Add the code interpreter capability — examples, formats and enablement](#)
- [Code interpreter security architecture — isolation, session lifecycle, network and quotas](#)
- [Overview of the Microsoft 365 Copilot orchestrator — how actions are chosen](#)
- [Microsoft Copilot architecture and how it works — the prompt path, grounding and post-processing](#)

Cowork is the Agentic Work capability taken to its conclusion: instead of asking Copilot for an artefact, you delegate an outcome and it executes over time. It is **grounded in Work IQ** — the same intelligence layer, not a connector stitching context together — it is **observable and controllable** at every step, and it is built in partnership with Anthropic, bringing Claude-class agentic capability into Microsoft 365. Every action still runs with your permissions and inside Microsoft's security, identity and governance framework.

The unit of work is different

COPILOT CHAT VERSUS COWORK

	COPILOT CHAT	COWORK
What it is	Always-on AI for drafting, summarising and answering	Agentic AI that completes multi-step work across Microsoft 365
Speed	Seconds to minutes	Minutes to hours, executing autonomously
Complexity	Single-step, single session	Multi-step workflows across tasks and sources
Use when	You need a quick draft, answer or insight	You need Copilot to take action across apps, files or systems
Typical scenario	Daily catch-up, project status, Q&A, drafting	Inbox and calendar clean-up, project launches, meeting preparation

COPILOT CHAT Click the composer — you ask, it answers, you iterate

Copilot

Chat Cowork

+ New chat

☐ Search

☐ Library

☐ Agents & Skills

☐ Notebooks

PINNED

☒ Researcher

☒ Analyst

☒ Sales

☒ Know Your Customer

Work IQ Auto

Summarise the Contoso account for me

Here are the key points from recent emails and meetings...
3 references

Draft ready — review and send when you're happy.
You act on the output

+ Message Copilot

Single-step, single session. Close the window and the work stops.

 Copilot

Chat

Cowork

New task

My tasks

Automations

Customize

PINNED

Weekly reset

Contoso account

Workshop prep

Weekly reset

Review last week, clean up my calendar, draft regrets for the conflicts and block three hours of focus time on Tuesday. Then prep a briefing for the Contoso meeting.

I've broken this into six steps and started work. I'll check in before anything is sent.

Task running · you can close this window

+ Message Cowork

Workspace

STEPS

6 / 6

Review last week's calendar

Identify conflicts and low-value meetings

Draft regrets — awaiting approval

Block focus time Tuesday

Pull Contoso emails and meeting notes

Produce the briefing document

OUTPUT

Contoso-briefing.docx

calendar-changes.xlsx

REFERENCES

Last week's meetings

Multi-step workflows across tasks and sources, with an approval gate in front of anything consequential.

Said plainly: **Chat helps you think through your work; Cowork gets the work done.** Chat generates output for you to act on. Cowork performs the coordinated actions itself — with an approval gate in front of anything that sends, posts, schedules or changes a shared system.

Three words that are not synonyms

"Conversation", "task" and "session" get used interchangeably, and they are not the same thing. Getting them straight is what makes the persistence story legible.

CONVERSATION · TASK · SESSION		
TERM	WHAT IT IS	LIFETIME
Conversation Copilot Chat	A turn-by-turn exchange. You ask, it answers, you iterate. The conversation is the work — close the window and the work stops.	Ephemeral as work. The thread is kept in your chat history, but nothing was executing.
Task Cowork	A delegated unit of work with its own state: <i>In progress, Needs your input, Complete, Failed</i> . Listed in the Tasks view, filterable, and openable at any time. Scheduled prompts and event-driven triggers are tasks too, under <i>Automations</i> .	Persistent. Survives a closed laptop; runs server-side; you come back to a finished outcome.
Session	The execution context a task runs in — the working environment with its input folder, output folder, loaded skills, granted permissions and progress log, all visible in the side panel. Selecting a task in the Tasks view opens its session.	Ephemeral. Session-scoped state does not carry to the next session.

What is ephemeral and what is persistent

The line falls in a specific place, and it falls there for a governance reason: **the compute is disposable, the output is enterprise content.**

Build Your Powerful AI Agents with Copilot · Track 2 workbook

11 / 56

THE PERSISTENCE BOUNDARY

EPHEMERAL — SCOPED TO ONE SESSION	PERSISTENT — OUTLIVES THE SESSION
The working environment itself, including intermediate scratch files	Output files , written to your OneDrive and SharePoint workspace — reachable from the side panel during the session and from your OneDrive Cowork folder forever after
Approval grants. "Always allow" and "don't ask again" apply <i>only to the current session</i> , and are reviewable or revocable from the Permissions section of the side panel	The task and its history — status, results and the conversation, all resumable from the Tasks view
Loaded skills. Cowork discovers your custom skills at the <i>start of each session</i> ; a skill added mid-session is not picked up until the next one	Custom skills , stored as SKILL.md files in OneDrive at /Documents/Cowork/skills/ — up to 50, each discovered automatically on the next session start
Plugin availability. Removing a plugin takes effect in the <i>next session</i> ; a session already in flight is not interrupted	Automations — scheduled prompts and event-driven tasks, with their own run histories, states and edit controls
The sensitivity label shown, which reflects the highest label across everything referenced <i>in that session</i>	Published artefacts — apps you publish and share by link, and documents that become enterprise knowledge the moment they are written

A useful test of whether you have understood the boundary: **losing your connection is not losing your work**. Cowork reconnects automatically and picks up where it left off, because progress belongs to the task, not to the browser tab you were watching it from.

UNDERSTANDING COWORK

Cowork is in this workbook as a **worked example of an agent**, not as a product tour. It is the clearest Microsoft-built illustration of the design decisions Sections 4 and 5 ask you to make yourself: a fixed set of skills that load only when a task needs them, an execution context that keeps nothing between sessions, output written somewhere durable, and an approval gate in front of every consequential action.

Read its shape as a reference design. The ephemeral-versus-persistent boundary above is the same boundary your own agent has to draw, and Cowork draws it where the compute is disposable and the output is enterprise content. When you reach 4.2 and find that a stateless sandbox will not hold your intermediate files, this is the model you are comparing against.

KNOWN LIMITS

Cowork cannot reach files stored locally on a device — it works in OneDrive and SharePoint. It cannot delete files or folders in either. It cannot read encrypted files even when you have access, and attached files must be under 200 MB. Microsoft does not validate user-authored custom skills, so their output needs the same review you would give a colleague's first draft.

SOURCES — SECTION 1.3

- [Use Copilot Cowork — sessions, files, approvals, skills](#)
- [Copilot Cowork common questions — the Chat-versus-Cowork table and limits](#)
- [Copilot Cowork: a new way of getting work done — the announcement](#)
- [Anthropic models in Microsoft Online Services](#)

THE EQUATION

Agent = LLM + Harness

An LLM on its own is not an agent. It generates text and stops. What turns it into something that performs work is the **harness** — and Microsoft Agent Framework says exactly that: an agent harness is *"the runtime scaffolding that turns a language model into an agent that can perform work."*

Reading the equation, one term at a time

The LLM is the reasoning and generation half. It interprets intent, plans in language, and writes the output. It has no memory of your last session, no ability to call a system, and no way to act on what it produces.

The harness is everything around it — *"a runtime that exists between the two: it determines when to call the model, what components to send it, interprets what comes back, and calls the right tools."* Concretely it:

- **Drives model and tool calls** — chooses when to invoke the model, and which tool to call with which arguments.
- **Manages conversation state and context** — sessions, history persistence, and context compaction when the window fills.
- **Applies approval policies** — standing approvals, auto-approval rules, and human-in-the-loop gates on tool execution.
- **Keeps the agent progressing** through a multi-step task — planning, todo tracking, and optional bounded re-invocation driven by evaluators or predicates.

Note what this means: **neither term of the equation is something you write.** You select a model and you select a harness. So where does the building happen?

Your design is the third thing — and it is built around the chosen harness

What you actually author is the **design layer**, and it has three components:

THE DESIGN LAYER The three things you actually author

I

Instructions

Define the agent's role, scope, tone and boundaries — and direct it on which resources to call, how to fill tool inputs, and how to respond.

K

Knowledge

The grounding sources the agent answers from — SharePoint, files, Dataverse, connectors, approved websites — so answers are anchored in your content rather than invented.

T

Tools

What the agent can do beyond talking — connectors, APIs, MCP servers, flows and other agents it can invoke.

All three sit **on top of the harness you choose** — never independently of it

This triad is Microsoft's own packaging, not a teaching device. Copilot Studio puts it plainly: *"All harnesses let you combine generative AI, knowledge, and actions to create modern agents built for how your teams actually work."* And the triad belongs to the agent rather than to whatever calls it — when a workflow hands a step to an agent, *"the agent's own instructions, knowledge, and tools are already configured wherever the agent was built."*

THE CONSEQUENCE THAT DRIVES THIS WHOLE WORKBOOK

The design layer is **built around the chosen harness** — never independently of it. The harness decides which of the three components you can express, how richly, and whether some are available at all. An 8,000-character instruction ceiling, whether tools can be orchestrated across steps or only called once, whether knowledge can be reasoned over or merely retrieved: all harness properties, inherited the moment you choose where to build.

Why the harness matters to you as a builder

The Copilot Studio guidance names four consequences, and each one is a design constraint you inherit the moment you pick a build surface:

WHAT THE HARNESS CHOICE DETERMINES

DIMENSION	WHAT CHANGES
How work gets done	Some harnesses follow the rules, branches and steps you define. Others take a goal, break it into steps, and adjust as they go.
How much automates end to end	A more capable harness handles longer multistep processes across tools, files and intermediate decisions — and recovers when a step fails.
What you get out of the box	File creation and editing, skills, memory, and tool-call orchestration are <i>harness</i> capabilities, not model capabilities.
How you are billed	Different harnesses use different pricing — user license, consumption, or Copilot Credits.

COMMON MISTAKE

Blaming the model for behaviour the harness owns. "The agent forgot what we discussed" is a state-and-context problem. "It stopped after one step" is an orchestration-loop problem. "It couldn't produce the Excel file" is a harness capability problem. Swapping the model fixes none of these.

SOURCES — SECTION 2

- [Harnesses in Copilot Studio — Microsoft Learn](#)
- [Agent Harness — Microsoft Agent Framework concepts](#)
- [Agents overview — Copilot Studio](#)
- [Add a Copilot node to a workflow — where an agent's instructions, knowledge and tools live](#)

Section 2 established that **Agent = LLM + Harness**, and that your design layer — instructions, knowledge, tools — is built around whichever harness you choose. Microsoft offers three, and they are not three tiers of one product. They are three different answers to the question *"what happens to my instructions, knowledge and tools at runtime?"*

THE SAME THREE DESIGN COMPONENTS, INTERPRETED DIFFERENTLY PER HARNESS

DESIGN LAYER	COPILOT CHAT	STANDARD	GITHUB COPILOT
Instructions	Decide which resources to call, fill tool inputs, shape the reply — capped at 8,000 characters	Set role and boundaries, but authored <i>topics</i> carry the actual conversation path	The primary control surface; the orchestration runtime reasons from them rather than executing a path
Knowledge	M365 grounding — SharePoint/OneDrive, connectors, email, people, Teams, web	Enterprise knowledge plus an existing prompt library	Grounding the agent <i>reasons over</i> across steps, not just retrieves
Tools	API plugins and MCP actions, called sequentially	Connector, custom, prompt and flow tools on authored paths	Connectors, MCP and connected agents, orchestrated with retry and recovery

Copilot chat harness — extend what people already use

The Copilot chat harness lets you **extend Microsoft 365 Copilot Chat**: connect enterprise knowledge so employees get grounded answers without leaving their everyday experience. It runs on current chat models and is built for connecting people to information — an onboarding agent answering from SharePoint is the canonical case. Publishing is to internal teams. Billing is consumption-based, or included in the Microsoft 365 Copilot user subscription license.

This is the harness most of your users meet first, because it is the one behind **Agent Builder** inside Microsoft 365 Copilot — plus pro-code declarative agents and declarative agents authored in Copilot Studio. There is no environment to provision, no solution to manage, and no publishing pipeline.

Design consequence: instructions are the whole steering mechanism, so the 8,000-character ceiling is a real design budget — and the reason skills exist (Section 3).

Standard harness — predictable, rule-based, authored

The standard harness is the classic Copilot Studio most makers already know: you define **topics, prompts and paths**, and the experience responds consistently for well-understood requests. It draws on your existing prompt library and enterprise knowledge, and it powers **agent flows**. Best fit: an internal help desk that answers common questions and routes simple requests through a workflow. Publishing reaches internal teams or external customers.

Design consequence: a fourth component joins the three — **topics**. Behaviour you would write as an instruction elsewhere, you author as a path here, and the runtime follows it rather than interpreting it.

GitHub Copilot harness — reasoning, files, recovery

The GitHub Copilot harness is the most capable option, built for reasoning-heavy agents and workflows that complete complex business processes. It became generally available on **3 August 2026**. Open copilotstudio.microsoft.com and this is the harness you are presented with first; the standard harness sits behind *Other ways to build*. That is placement, not a lock-in — Microsoft is explicit that the two run alongside each other and *"you choose which harness to use when you create a new agent."* Instead of following a fixed script it takes a goal, breaks it into steps, calls tools across connectors, knowledge, MCP and connected agents, and **adjusts when a step fails**. It natively creates and edits Word, Excel, PowerPoint and PDF files, supports **skills and memory**, and runs each task in a secure sandbox governed by Copilot Studio. Usage is billed in **Copilot Credits** — including build and test, not just runtime.

Design consequence: you write fewer mechanics and more intent. Instructions state the goal and the guardrails; the harness supplies the plan, the retries and the recovery you would otherwise have had to author yourself.

THREE HARNESSES, THREE APPROACHES			
CONSIDERATION	COPILOT CHAT	STANDARD	GITHUB COPILOT
Best for	Extending M365 Copilot Chat with enterprise knowledge	Rule-based agents and structured conversations	Complex, multi-step business processes
How it works	Connects enterprise knowledge to M365 Copilot Chat	Follows the topics and rules you define	Reasons through a goal on its own, step by step
You build it in	Agent Builder (and declarative agents in Copilot Studio)	Copilot Studio	Copilot Studio
Recovers from problems	Not a focus	Follows the paths you built	Retries and finds alternative paths automatically
Works with files	Not a focus in the harness — but it generates Word, Excel, PowerPoint and PDF through code interpreter	Not a focus in the harness — but a tool can do it : a connector action or agent flow (SharePoint, OneDrive, Excel Online) creates or edits the file. It is deterministic — you map the target file, the location and the change yourself, and the agent only triggers it	Creates, edits and reasons over Word, Excel, PowerPoint and PDF natively — no wiring
Skills & memory	Skills in preview; no memory	Not a focus	Yes — both, natively
Publishing	Internal teams	Internal teams or external customers	Internal teams or external customers
Billing	M365 Copilot Basic/Premium License	Standard-harness licensing	Copilot Credits (usage-based, including build and test)

PICK YOUR HARNESS What each one is built to do

01 Copilot chat harness Extending M365 Copilot Chat with enterprise knowledge HOW IT WORKS Connects enterprise knowledge to M365 Copilot Chat YOU BUILD IT IN Agent Builder, and declarative agents in Copilot Studio FILES Generates Word, Excel, PowerPoint and PDF through code interpreter SKILLS & MEMORY Skills in preview; no memory PUBLISHING & BILLING Internal teams · M365 Copilot Basic/Premium License	02 Standard harness Rule-based agents and structured conversations HOW IT WORKS Follows the topics and rules you define YOU BUILD IT IN Copilot Studio FILES A tool can do it — a connector action or agent flow, deterministic and mapped by you SKILLS & MEMORY Not a focus PUBLISHING & BILLING Internal teams or external customers · Standard-harness licensing	03 GitHub Copilot harness Complex, multi-step business processes HOW IT WORKS Reasons through a goal on its own, step by step YOU BUILD IT IN Copilot Studio FILES Creates, edits and reasons over Word, Excel, PowerPoint and PDF natively — no wiring SKILLS & MEMORY Yes — both, natively PUBLISHING & BILLING Internal teams or external customers · Copilot Credits, usage-based
---	---	--

Recovery from problems is the dividing line: not a focus on chat, the paths you built on standard, retries and alternative paths on GitHub Copilot.

*The harness you pick is inherited, not negotiated: instruction ceilings, whether tools can be orchestrated across steps, and whether knowledge is reasoned over or merely retrieved are all harness properties. Read "not a focus" as **the harness won't do it for you** rather than impossible — on the standard harness, files are a **wiring** job you author; on the GitHub Copilot harness they are a **capability** you inherit.*

PLANNING RULE

Because the design layer is built around the harness, **choose the least capable harness that still clears the requirement** — then design instructions, knowledge and tools to fit it. Capability you don't need is cost, governance surface and latency you do pay for. Section 4.2 works this through against a real design.

SOURCES — SECTION 2.1

- [Agents overview — three harnesses, three approaches](#)
- [Harnesses in Copilot Studio — comparison table](#)
- [Overview of usage-based billing — Copilot Credits](#)
- [Licensing for agents powered by the standard harness](#)
- [What's new in Copilot Studio — the GitHub Copilot harness runs alongside the standard harness](#)

SHORT ANSWER

No. The harness does not change. Agent Builder still produces *declarative agents running on the Copilot chat harness*. Custom skills are a new *component inside the declarative agent* — not a migration to the GitHub Copilot harness, and not a new runtime. What changes is how you package instructions, not who runs the loop.

What's currently in Microsoft 365 Copilot Agent Builder

- **Custom skills in declarative agents (preview).** Available to organisations in the **Microsoft Frontier Preview**, added either in Agent Builder (upload a `.zip` package) or with the Microsoft 365 Agents Toolkit (import the skill directory).
- **Skills can be authored from a description.** In Copilot chat you can ask Agent Builder to create a reusable skill in natural language, then review it under *Configure* → *Skills*.
- **They replace an older meaning of the word.** Earlier guidance used "skills" to mean task descriptions written *inline in the agent's instructions*. Custom skills replace that approach with a separately packaged component the agent references.
- **Agent Builder usability updates (2026).** Refreshed side-by-side *Describe* and *Configure* panes with a *Try it* toggle for testing (rolled out from late April 2026); newly created agents get **capabilities enabled by default**, a simplified knowledge configuration experience, and **"Uploaded Files" renamed to "Attachments"** (late August 2026). None of these changed how published agents work, admin controls, or existing agents.
- **Adjacent additions:** AI-generated agent icons, new templates, additional knowledge sources (People, OneNote), and a *Copy to Copilot Studio* path when an agent outgrows the builder.

What a maker sees, by license tier

AGENT BUILDER The same builder, two different sets of options

Copilot Chat (Basic)

Microsoft 365 Copilot (Premium)

Same page, fewer sections: **no Skills card** and **no Work content**. Knowledge is the sources you add plus, if you switch it on, the public web.



New Agent

Describe your agent

Auto ▾

Instructions ⓘ

Suggest improvements

Describe what this agent should do, define its tone, and outline any rules or guidelines it must follow

Knowledge ⓘ

Suggest improvements

Focus your agent by adding specific knowledge sources

+ Add knowledge

☒ Web search ☒

Suggested prompts ⓘ

Title	Message
Enter a title	Enter a message
Enter a title	Enter a message

+ Add a suggested prompt

WHAT PREMIUM ADDS

- **Skills** card (preview) — absent on Basic.
- **Work content** switches under Knowledge — Cloud files, Outlook, Teams and Copilot connectors.

Both licenses share the agent header, **Instructions**, **Knowledge** with **Web search**, and **Suggested prompts** — and on both, every agent can write code and create Word, Excel and PowerPoint files.

The availability of knowledge sources and capabilities differs based on the user's license. Memory did not come along with skills — neither tier has it.

Scroll **Configure** to set the agent up end to end — name, **Instructions**, **Skills**, **Knowledge** with its **Work content** switches, and **Suggested prompts**.

New Agent

Describe your agent

Auto ▾

Instructions ⓘ

Suggest improvements

Describe what this agent should do, define its tone, and outline any rules or guidelines it must follow

Skills ⓘ **PREVIEW**

+ Add

1

rfp-requirements.zip

2

response-check.zip

3

report-render.zip

Knowledge ⓘ

Suggest improvements

Focus your agent by adding specific knowledge sources

+ Add knowledge or just drop files here

Work content ⓘ

Cloud files

☒

Outlook

☒

Teams

☒

Copilot connectors

NONE ☒

Web search

☒

Attachments ⓘ

NONE

Suggested prompts ⓘ

Title	Message
Enter a title	Enter a message
Enter a title	Enter a message

+ Add a suggested prompt

WHAT PREMIUM ADDS

- **Skills** card (preview) — absent on Basic.
- **Work content** switches under Knowledge — Cloud files, Outlook, Teams and Copilot connectors.

Both licenses share the agent header, **Instructions**, **Knowledge** with **Web search**, and **Suggested prompts** — and on both, every agent can write code and create Word, Excel and PowerPoint files.

The availability of knowledge sources and capabilities differs based on the user's license. Memory did not come along with skills — neither tier has it.

BOUNDARIES THAT STILL BIND

A skill's scripts run in a secure sandbox with **no network access**, **no package installation**, and **no authenticated network calls**. Scripts cannot reach connectors, API plugins or MCP servers through the sandbox — only the orchestrator can use those. The declarative agent's own sequential grounding model is unchanged, so **iterative reasoning loops and chained operations still are not available** on this harness.

SKILL LIMITS IN AGENT BUILDER VS AGENTS TOOLKIT (PREVIEW)

ITEM	AGENT BUILDER	AGENTS TOOLKIT
Maximum skills per agent	8	8
Package format	Compressed <code>.zip</code> , max 50 MB	Skill directory; <code>.zip</code> not supported
Maximum file size	25 MB per file	No skill-level limit; app package capped at 10 MB
Maximum files	350 files across all skills	
Directory depth	Maximum 3	
Reuse across agents	Not supported at this stage of the preview	
Skills + embedded files	Cannot be combined yet — support planned	

Agent Builder (Premium user) vs the Copilot Studio GitHub Copilot harness

Both surfaces now offer skills, and the skill artefact looks the same on both — a `SKILL.md` with optional resources and scripts. That surface similarity is exactly why the two get conflated. The differences that matter sit underneath it.

THE TWO BUILD SURFACES, COMPARED

DIMENSION	AGENT BUILDER — M365 COPILOT (PREMIUM)	COPILOT STUDIO — GITHUB COPILOT HARNESS
What you produce	A declarative agent on the Copilot chat harness	An agent on the GitHub Copilot harness, with its own orchestration runtime
Skills	Yes — preview, Frontier-gated. Upload the complete <code>.zip</code> ; a bare <code>SKILL.md</code> is rejected. Maximum 8 per agent	Yes — native. Upload a <code>SKILL.md</code> or a packaged <code>.zip</code> ; described as portable packages on an open specification
Skill reuse	Not supported at this stage of the preview — each agent needs its own copy	"Create a skill once and add it to multiple agents"
Memory	No agent-level memory feature. The declarative agent manifest has no memory property	Memory (preview) — a maker toggle in the Build tab, on or off per agent
File persistence	Code interpreter output is downloadable <i>during the session only</i> and is not persisted afterwards	Files the agent creates are stored with the conversation — referenceable across turns, 10 MB per file, deleted 28 days after the conversation's last activity
Files	Generates Word, Excel, PowerPoint, PDF via code interpreter	Creates, edits and reasons over the same formats natively
Recovery	Sequential grounding and tool calls; no iterative loop	Goal decomposition, retries, alternative paths
Publishing	Internal teams, inside Microsoft 365 Copilot	Internal or external channels — including back into Microsoft 365 Copilot's agent list

Memory and workspace — what the words actually mean

This is the distinction most often stated loosely, so it is worth being precise about both halves.

Memory is part of Copilot Studio feature. On the GitHub Copilot harness, *Memory* is a preview capability the maker switches on per agent: *"Memory in Copilot Studio helps an agent remember details from its interactions and use that context for future interactions."* Its scoping is the important part — **each agent keeps a separate memory store for every user**, a user's memories are private to that user and invisible to the maker; memories are deleted after **28 days** without interaction, and memory is disabled in group chats and Teams channels. Turning the toggle off stops the agent using memories but does not delete them.

Agent Builder has no equivalent. There is no memory toggle, and the declarative agent manifest carries no memory property. Worth separating from a similar-sounding feature: Microsoft 365 Copilot has its own *personalisation memory* for Copilot Chat users, stored in the user's mailbox — that is a property of the Copilot experience, not something your declarative agent can configure or reach.

Licensing — the difference that usually decides it

The two surfaces are not two prices for the same thing. They are two billing models, and the second one bills earlier than people expect.

HOW EACH IS PAID FOR		
	AGENT BUILDER — M365 COPILOT (PREMIUM)	COPILOT STUDIO — GITHUB COPILOT HARNESS
To build	Included. <i>"Your Microsoft 365 Copilot license includes the agents you build by using Agent Builder"</i>	Maker license is \$0 , but credits are consumed from the moment you start building — authoring, previewing, testing and generating evaluations all charge
To run, for a licensed user	No extra charge. Generative answers and tenant Graph grounding are zero-rated for M365 Copilot license holders	Still billed. <i>"Creation and runtime activities consume Copilot Credits regardless of the user's license"</i>
To run, for an unlicensed user	Instruction- and public-website-grounded agents are free; agents touching shared tenant data are metered in Copilot Credits and are off by default	Billed the same way — usage is attributed at environment and agent level, not per user
Billing unit	Copilot Credits, only when metered consumption applies	Copilot Credits, covering LLM tokens, tools and the harness itself

Two consequences follow, and both are easy to miss. First, **the \$30 Microsoft 365 Copilot license includes Copilot Studio use rights for the standard harness, not the GitHub Copilot harness** — and that included usage is bounded by fair-use limits Microsoft does not publish a number for. Second, publishing a GitHub Copilot harness agent *into* Microsoft 365 Copilot does not make it free to the people using it: the license that zero-rates a declarative agent does not zero-rate this one.

HOW TO CHOOSE

Build in **Agent Builder** when the work is grounded Q&A plus packaged procedure, for people who already hold a Copilot license — the marginal cost is zero and the governance is inherited. Move to the **GitHub Copilot harness** when you genuinely need memory across conversations, file editing, or multi-step recovery — and price it, because credits start accruing during the build, not at go-live.

TAKE THIS INTO PLANNING

Skills raise the *ceiling of the Copilot chat harness* — longer, more reliable, more repeatable instruction sets — without moving the *walls*. If your requirement fails on the walls (memory, loops, transactions, file editing and persistence, external publishing), skills will not rescue it; a harness change — and that change has a different billing model attached, not just a different feature list.

SOURCES — SECTION 2.2

- [Custom skills in declarative agents \(preview\) — limits, sandbox, governance](#)
- [Add custom skills to your declarative agent in Agent Builder \(preview\)](#)
- [Agent Builder in Microsoft 365 Copilot — governance, data, limitations](#)
- [Copy an agent to Copilot Studio](#)
- [Get started with the Microsoft Copilot Frontier Program — enrolment prerequisite](#)
- [Memory in Copilot Studio \(preview\) — per-user scoping, 28-day expiry, maker toggle](#)
- [Agent skills on the GitHub Copilot harness — portable packages, reuse across agents](#)
- [Files created by an agent — conversation-scoped retention](#)
- [Overview of usage-based billing — credits charged from the moment you build](#)
- [Manage usage for GitHub Copilot harness agents — what an M365 Copilot license does not cover](#)
- [Cost considerations for Microsoft 365 Copilot extensibility](#)

Same word, three different bills of materials. Knowing which parts exist on which harness is what stops a team designing a component that its runtime cannot host.

A. Copilot chat harness — the declarative agent

A declarative agent is *declared configuration loaded into Microsoft 365 Copilot* — no custom orchestrator, no custom model. What ships is an **app package**:

```
agent-package.zip
├─ manifest.json           # Microsoft 365 app manifest           (required)
├─ declarativeAgent.json  # name, description, instructions,
│                          # capabilities, conversation starters,
│                          # actions, behavior overrides         (required)
├─ ai-plugin.json         # API / MCP plugin manifest           (only if actions)
├─ color.png, outline.png # agent icons                         (required)
└─ skills/               # custom skills, preview              (optional)
    └─ my-skill/SKILL.md
```

DECLARATIVE AGENT — COMPONENT INVENTORY (MANIFEST V1.8)

COMPONENT	WHAT IT IS	LIMIT / NOTE
name / description	Identity surfaced to users	100 / 1,000 characters
instructions	Behaviour, scope, tone, what to avoid	8,000 characters — the ceiling skills exist to work around
capabilities	SharePoint & OneDrive, web search, code interpreter, image generation, email knowledge, people, Teams messages, Teams meetings, Dataverse, Copilot connectors, scenario models — and now email and meeting <i>actions</i>	One of each derived type
conversation starters	Example prompts on the agent's front door	Maximum 12
actions	API plugins and MCP servers the agent may call	1–10 plugin objects
skills (preview)	Packaged, reusable task modules	Maximum 8 per agent
behavior overrides / disclaimer	Behaviour configuration and start-of-chat disclaimer text	Optional
Orchestrator + model	Microsoft-managed	Not under the builder's control

The shape of the data flow matters as much as the parts. "Because Microsoft grounding and calls to external tools happen sequentially, you can't use chained operations with grounding data or looped operation plans." Design for "one grounding pass plus one tool call produces a complete answer" — not for interdependent multistep workflows.

B. Standard harness — the authored agent

Here you are assembling a conversation design, not just a configuration. The building blocks:

STANDARD HARNESS — COMPONENT INVENTORY

COMPONENT	WHAT IT CONTROLS
Instructions	Role, scope, priorities, behavioural boundaries
Primary model	The model used for orchestration and generation
Knowledge	Grounding sources — SharePoint, Dataverse, files, approved websites
Topics	Curated conversational paths, built from trigger phrases and nodes
Tools (actions)	Connector, custom, prompt and flow tools the agent can invoke
Agent flows	Multistep deterministic automation, callable by the agent or a trigger
Triggers	Events that start work with no user prompt
Variables	Values gathered and passed through a conversation
Channels	Where the published agent lives — Teams, M365 Copilot, a website, custom apps
Analytics & activity	Usage, resolution rate, failed actions, execution traces

The design question on this harness is always the same: *where should probabilistic AI stop and deterministic logic begin?* Use the model to interpret intent and draft; use a topic, flow or API to validate data, enforce permissions and execute the transaction.

C. GitHub Copilot harness — the reasoning agent

Configured from the **Build** tab (the chat harness uses *Configure*). Instructions are the primary mechanism for controlling behaviour — the enhanced orchestration runtime interprets them rather than executing an authored path.

GITHUB COPILOT HARNESS — COMPONENT INVENTORY

COMPONENT	WHAT IT ADDS OVER THE OTHER HARNESSES
Name, icon, description	Identity; icon catalogue with colour and accent
Instructions	Primary behaviour control — purpose, tone, boundaries, refusals, escalation and handoff triggers
Knowledge	Grounding sources, reasoned over rather than merely retrieved
Tools	Connectors, MCP servers and connected agents , orchestrated across steps
Skills	First-class, natively supported — packaged capability modules
Memory	Context retained across a task and beyond a single turn
File handling	Native creation and editing of Word, Excel, PowerPoint and PDF
Sandbox	Each task executes in a secure sandbox governed by Copilot Studio
Planning & recovery	Goal decomposition, retries, alternative paths when a step fails
Billing surface	Copilot Credits, consumed during build, test and run

SOURCES — SECTION 2.3

- [Declarative agent architecture — components, data flow, limits](#)
- [Declarative agent manifest schema v1.8](#)
- [Configure agent details and instructions \(chat vs GitHub Copilot harness\)](#)
- [Write agent instructions — Copilot Studio](#)
- [Agent flows overview — the standard-harness automation component](#)
- [Custom skills in declarative agents \(preview\) — the 8-skill limit and manifest 1.9](#)

Skills — what they are and how they are built

A **skill** is a modular capability you add to an agent so it performs a specific task more reliably and repeatably. Not more knowledge, and not another tool: a packaged *procedure*, with the assets and scripts that procedure needs travelling alongside it.

THE ONE-LINE DISTINCTION

An agent is the "who". A skill is the "how". Add an *agent* when there is a new responsibility. Add a *skill* when there is a specific task that responsibility must perform the same way every time.

The anatomy of a skill

A skill is a file directory. Exactly one file is mandatory.

```
my-skill/
├── SKILL.md                # REQUIRED
│   ├── YAML front matter  # name + description → how the model decides to use it
│   └── instructions       # the procedure, under 20,000 characters
├── <resource files>       # optional – templates, reference data, definitions
└── <scripts or folders>   # optional – .py .js .mjs .cjs .ts .mts .sh .bash
                           # max directory depth 3
```

- **SKILL.md** holds the instructions that direct the model to complete the task. Its front matter carries the **name and description** — these are what the orchestrator reads to decide when the skill applies, so they are load-bearing, not labels.
- **Resource files** are the assets the task needs: branded templates, KPI definitions, reference tables, schemas.
- **Scripts** replace model-generated code with code you control — the same transformation, the same chart, every run.

A worked example from the documentation: a finance team builds a *quarterly business review* skill that bundles the instructions for producing the review, scripts that transform data and generate charts, and assets such as financial models, KPI definitions and branded slide templates. The agent reaches for the skill when a user asks for that task.

Why skills instead of longer instructions

THE FOUR STATED BENEFITS

BENEFIT	MECHANISM
Optimised context, better accuracy	<i>Progressive disclosure</i> — the model loads a skill's instructions and contents into context only when the task requires them. Context stays focused as the skill library grows.
Scale instructions without quality loss	Decompose complex guidance into modular skills; give longer, more specific direction without degrading instruction-following and without hitting the 8,000-character declarative-agent instruction limit.
Package reusable assets	Instructions, resources and scripts ship together as one integrable capability.
Enterprise-ready	Scripts run in a secure sandbox; sensitivity labels on skill files are retained and honoured; admin governance available; files stored in tenant-scoped SharePoint Embedded containers.

What a skill may contain

SUPPORTED FILE AND SCRIPT TYPES

CATEGORY	TYPES
Instructions & resources	<code>.md</code> <code>.txt</code> <code>.rtf</code> <code>.json</code> <code>.xml</code> <code>.yaml</code> <code>.yml</code> <code>.ini</code> <code>.config</code> <code>.utf8</code> <code>.log</code> <code>.docx</code> <code>.doc</code> <code>.docm</code> <code>.pdf</code> <code>.ppt</code> <code>.pptx</code> <code>.ppsm</code> <code>.xlsx</code> <code>.xls</code> <code>.xslm</code> <code>.csv</code> <code>.tsv</code> <code>.html</code> <code>.htm</code> <code>.png</code> <code>.jpg</code> <code>.jpeg</code> <code>.gif</code> <code>.bmp</code>
Scripts & binaries	<code>.py</code> <code>.js</code> <code>.mjs</code> <code>.cjs</code> <code>.ts</code> <code>.mts</code> <code>.sh</code> <code>.bash</code>

SANDBOX LIMITS — DESIGN AROUND THESE

No internet or network access at runtime. No package installation. No authenticated network calls. A skill may use packages already present in the sandbox — but do not depend on one unless its availability is confirmed. Connectors, API plugins and MCP servers remain available to the agent *through the orchestrator*, but scripts cannot call them from inside the sandbox. Sensitivity labels are honoured; user-defined permissions and Double Key Encryption are not supported.

SOURCES — SECTION 3

- [Custom skills in declarative agents \(preview\) — full reference](#)
- [Add custom skills in Agent Builder — package and upload](#)
- [Write effective instructions for declarative agents — how instructions reference skills](#)

SECTION 4

Planning a Document Comparison Agent

Sections 1 to 3 covered the platform, the runtime and the component. This section is about the step between understanding those and opening a builder: **turning a way of working into an agent design**. A team's framework — how they compare, what they check, what counts as a finding — is not yet a specification. Section 4 is the translation: deciding what must be deterministic and what needs judgement, what becomes a skill and what stays an instruction, and which harness the result belongs on. Work through it with your own scenario in mind; **Section 5 then builds one end to end**.

THE SCENARIO

Everybody has their own framework for comparing documents. Legal wants clause-by-clause: does clause 7.2 in the counterparty's draft still say what our template says? Procurement wants conformance: does this contract satisfy our policy? Risk wants both, then wants the exceptions ranked. The documents are similar; the *frameworks* are not.

The instinct is to write the framework into the agent's instructions and let the model compare. That fails three ways, and the failures are instructive enough to design around.

4.1.1 · Why "just let the model compare it" does not hold

THREE FAILURE MODES OF INSTRUCTION-ONLY COMPARISON

FAILURE	WHAT HAPPENS	WHY IT IS STRUCTURAL
Budget	A real comparison framework — level definitions, matching rules, severity model, exception handling — does not fit alongside the agent's other instructions.	Declarative agent instructions cap at 8,000 characters . This is the exact pressure skills were built to relieve (Section 3).
Completeness	Ask a model to align 400 clauses and it will produce a confident, well-written table that silently omits some of them. Nobody can tell which.	There is no mechanism in a language response that <i>guarantees</i> every unit was considered. Alignment is a set operation; prose is not.
Traceability	A reviewer asks "where did this finding come from?" and the answer is a paraphrase.	Comparison output is evidence. Evidence needs a stable identifier pointing at a specific unit of a specific document — not a restatement.

The reframe that makes this buildable: document comparison is mostly a *deterministic* problem wearing a judgement hat. Parsing, identifying, aligning and mechanical rule-testing are all script work with exactly one right answer. Only a thin top layer — *is this change material? is this clause acceptable?* — genuinely needs a model. Sorting which is which is the entire design, and it is what the skills exist to encode.

4.1.2 · The determinism split

WHO DOES WHAT, AND WHY

TASK	OWNER	REASONING
Parse a document into units and assign IDs	Script	Same input must always give the same units. A drifting parse invalidates every downstream ID.
Align unit A↔B across two documents	Script	A set operation with a provable completeness property.
Detect that text changed, and by how much	Script	String comparison. A ratio is a fact.
Evaluate a mechanical rule — presence, threshold, forbidden phrase, cross-reference	Script	Two reviewers with a ruler would agree. That is the test.
Judge whether a detected change is <i>material</i>	Model	Requires reading meaning, not characters.
Evaluate a judgement rule — "is this indemnity balanced?"	Model	Genuinely interpretive; needs a rubric and an evidence quote to stay reviewable.
Write the narrative and the recommendation	Model	What models are for.
Render the report	Script	Consistent artefacts every run; no formatting drift between sessions.

THE RULE OF THUMB

If two competent reviewers working from the same documents would reach the *identical* answer, it belongs in a script. If they might reasonably differ, it belongs to the model — and then it needs a rubric, a severity, and a quoted piece of evidence so the disagreement is visible rather than buried.

4.1.3 · What the agent handles — two comparison cases

"Compare these documents" covers two distinct jobs. Supporting both is worth the effort; supporting more than both is where this design starts to strain.

THE TWO SUPPORTED CASES

	CASE A — TWO VERSIONS	CASE B — ONE BASELINE, MANY CANDIDATES
The question	"What changed between v6 and v7, and does any of it matter?"	"Which of these thirty drafts deviate from our template, and which deviate worst?"
Documents	2 — a baseline and a candidate	1 baseline + up to 5 candidates
Alignment	One pairwise pass	Each candidate aligned to <i>the baseline</i> , then joined
The deliverable	A clause-by-clause change table	A matrix — one row per baseline clause, one column per candidate — plus a ranking of candidates by severity count
Who reads it	The person negotiating that document	Someone triaging a stack, who will open two of them properly

Case B uses a spine, not a mesh. Comparing every document against every other is $O(n^2)$ and produces a table nobody can read. Instead the baseline is the **spine**: each candidate is aligned to it in a separate pairwise pass, and the results join on baseline unit ID. That keeps the work linear and the output legible.

The spine has one honest cost, and the design must state it rather than hide it: **a clause that appears in two candidates but not in the baseline has no row to sit in**. Those go to an explicit *unmatched* section, and the completeness receipt generalises to cover them — *every unit of every document appears exactly once, either in a matrix row or in the unmatched list*.

WHY FIVE CANDIDATES

The ceiling is set by **context**. Scripts can align far more than five documents; what cannot scale is the model's judgement pass afterwards, which has to read the modified rows through conversation context. At roughly 400 clauses per document with 15% modified, five candidates already puts several hundred rows in front of the model. **Six documents total is the design ceiling; three candidates is the comfortable default.** Treat both as numbers to validate against your own documents rather than as facts — measure with a real set early, and lower the ceiling if judgement quality degrades before you reach it.

Beyond that ceiling the shape of the problem changes. Thirty contracts processed nightly is a batch pipeline wearing an agent costume — the right answer there is an agent flow or a scheduled process calling the same logic, with the agent as the *interface to results* rather than the engine. That is a harness question, and it belongs in Section 4.2.

A CASE THIS DESIGN DOES NOT COVER

Cross-document *consistency* — does the SOW contradict the MSA? does the DPA cover the data the SOW describes? — arrives disguised as a comparison request but is a different problem. Those documents are not versions of each other and share no structure to align, so `clause-align` is the wrong primitive entirely. The operation there is *extract obligations and test for conflict across documents*. It would reuse `document-extract` and the rule engine, and skip alignment altogether. Recognise the request; do not try to serve it with a spine.

4.1.4 · The skill set

Five skills, against a budget of eight. The spare capacity is deliberate — leave room for the skill you discover you need in testing.

PROPOSED SKILLS AND THEIR CONTRACTS

SKILL	RESPONSIBILITY	PRODUCES	WEIGHT
<code>comparison-intake</code>	Establish <i>how</i> this comparison must be done, and confirm it	<code>run-manifest.json</code>	Instruction-heavy, no script
<code>document-extract</code>	Parse documents into hierarchically identified units	<code>units.json</code>	Script + profile resources
<code>clause-align</code>	Match units across documents and classify change	<code>alignment.json</code>	Script
<code>rule-check</code>	Test units against a guideline pack	<code>findings.json</code>	Script + model, rule pack resources
<code>comparison-report</code>	Render the reviewable artefact	<code>.xlsx</code> / <code>.docx</code>	Script + templates

Notice that every skill's contract is **a named artefact with a schema**, not "help the user with X". That is what makes the chain inspectable — and, as 4.2.1 explains, what makes it survivable on a runtime that keeps no state between script runs.

✓ 4.1.4.1 · Skill 1 — `comparison-intake`

Your first skill: *understand how the comparison needs to be done*. The design trap is letting it end in a conversation. It must end in **a machine-readable manifest that the other skills consume** — the conversation is how you get there, not the output.

```
{
  "mode": "baseline_vs_many",    // versions | baseline_vs_many | doc_vs_rules | both
  "documents": [
    { "id": "A", "role": "baseline", "file": "MSA-template-v7.docx" },
    { "id": "B", "role": "candidate", "file": "contoso-redline.docx" },
    { "id": "C", "role": "candidate", "file": "fabrikam-redline.docx" }
  ],
  "profile": "numbered-legal",   // which extraction profile
  "profile_basis": "detected",   // detected | user-specified – see 4.2.2
  "rule_pack": "procurement-v3", // null when no rule checking
  "scope": { "include_paths": ["4", "7", "9"], "exclude_labels": ["annex"] },
  "materiality": { "ignore_whitespace": true, "min_similarity": 0.98 },
  "severity_model": ["blocker", "major", "minor"],
  "output": ["xlsx", "docx"]
}
```

The skill's instructions should do four things: ask only for what it cannot infer from the attached files; propose defaults rather than presenting a blank form; **show the completed manifest back and require confirmation before any run**; and refuse to proceed if `mode` and `rule_pack` contradict each other. Intake is also where you capture the framework's vocabulary — if this team calls them "articles" rather than "sections", the manifest carries that through to the report.

Both cases from 4.1.3 live in this one manifest. `documents` is a list with explicit roles rather than a fixed baseline-and-candidate pair, so two versions is simply the one-candidate case. That is the change that makes multi-document work without a second intake path. Intake also owns two new guards: reject a run with more than five candidates and explain the ceiling, and reject a `baseline_vs_many` run where no document carries `role: baseline`.

✓ 4.1.4.2 · Skill 2 — `document-extract`, and how to make hierarchy flexible

This is the question that decides whether the agent serves one team or twenty. **The answer is not to write a cleverer parser. It is to stop encoding structure in code at all.**

Put the structure in a **profile** — a resource file in the skill package that declares, per level, what that level looks like. The script reads the profile and applies it. Supporting a new framework becomes *adding a file*, not editing logic or building a second skill.

```
name: numbered-legal
levels:
  - level: 1
    label: section
    match: '^(?P<num>\d+)\.\s+(?P<heading>.+)$'
  - level: 2
    label: subsection
    match: '^(?P<num>\d+\.\d+)\s+(?P<heading>.+)$'
  - level: 3
    label: point
    match: '^\((?P<num>[a-z])\)\s+(?P<text>.+)$'
id:
  format: dotted          # produces 4.2.c
  inherit_parent: true
fallback:
  unmatched: append_to_previous
  synthetic_id: '{parent}.p{n}'
```

Every unit the script emits uses one canonical record, regardless of which profile produced it. Downstream skills only ever see this shape:

```
{
  "uid":      "A:4.2.c",
  "doc":      "A",
  "path":     "4.2.c",
  "level":    3,
  "label":    "point",
  "heading":  "Termination for convenience",
  "text":     "Either party may terminate ...",
  "parent":   "A:4.2",
  "order":    128,
  "hash":     "sha1:9f2c...",
  "loc":      { "para": 412 }
}
```

FOUR DECISIONS THAT MAKE THE EXTRACTOR FLEXIBLE

DECISION	WHY IT MATTERS
Prefer the document's own numbering for IDs	A reviewer can find 4.2.c in the PDF on their desk. A synthetic index they cannot locate destroys the traceability you built the agent for.
Synthesise IDs only as fallback, and carry a content hash	Unnumbered documents still need stable handles, and the hash lets you tell "this unit moved" from "this unit changed" across re-runs.
Ship three profiles, not one	A numbered-legal profile, a Word heading-style profile, and a plain-paragraph fallback cover most of what arrives. Named profiles also give intake something concrete to offer.
Add a detect mode	The script samples the opening paragraphs, reports which candidate patterns matched and how often, and the model proposes a profile for the user to confirm. Discovery stays deterministic; only the proposal is generative.

TWO CONSTRAINTS THAT SHAPE THE SCRIPT

Parse .docx with the standard library. A .docx is a zip containing word/document.xml — zipfile plus xml.etree reads paragraphs, styles and numbering with no third-party dependency. That matters because the sandbox has **no network access and cannot install packages at runtime**, and Microsoft's own guidance is not to depend on a package unless its availability is confirmed. PDF is the weak case: prefer .docx inputs where you can, and let Copilot's own file reading carry PDFs when you cannot.

A profile discovered at runtime cannot save itself. Skill packages are immutable at run time. detect can propose a profile and hand the user the YAML, but persisting it is a package update by the maker. Design for two paths: profiles shipped in the package, and a one-off profile pasted into the prompt for a single run.

✓ 4.1.4.3 · Skill 3 — clause-align

Alignment is where the completeness guarantee is won or lost, so it is pure script. A three-pass match handles nearly everything: **match on ID** where both documents share a numbering scheme; fall back to **heading similarity**; fall back again to **body-text similarity** (difflib.SequenceMatcher is standard library, which keeps the dependency promise). Each aligned pair gets a status — unchanged, modified, moved, added, removed — plus a similarity ratio.

THE COMPLETENESS RECEIPT

Have the script assert, and print, that **every unit in A and every unit in B appears exactly once in the alignment table**, and fail loudly if not. This single line is the thing an instruction-only agent can never offer: a verifiable claim that nothing was skipped. Put the count in the report header — "412 units in baseline, 428 in candidate, 412 aligned, 16 added, 0 unaccounted" — and the reviewer's first question is answered before they ask it.

The model's role starts only *after* this table exists: it reads the modified rows and judges materiality. It never decides what is on the list.

✓ 4.1.4.4 · Skill 4 — rule-check, and where guidelines should live

Should the guidelines be attached as a knowledge source? The question has a two-part answer, because "the guidelines" are usually two different artefacts wearing one name: **the rules you test against**, and **the policy those rules come from**. They belong in different places.

WHERE EACH KIND OF GUIDELINE BELONGS

ARTEFACT	WHERE IT LIVES	WHY
The machine-readable rule pack — testable assertions with thresholds, patterns and severities	A resource file inside the skill package	It is versioned with the logic that executes it, it is read deterministically rather than retrieved, and there is no chance the retrieval step returns the wrong chunk of it. Rules are code-adjacent, not literature.
The authority document — the actual policy, standard or playbook the rules derive from	A SharePoint / OneDrive knowledge source	Humans maintain it, it changes on its own cadence, and the agent needs it to answer "why is this a blocker?" with a citation rather than a restatement.

So: **rules in the package, policy in SharePoint, nothing embedded**. The rule pack carries an authority pointer on every rule so a finding can cite the clause of the policy it came from, and the knowledge source is what lets the agent quote that policy when challenged.

Designing the rule pack

Every rule declares which engine evaluates it. That field is the determinism split from 4.1.2, made explicit per rule:

```
pack: procurement-v3
authority: "Procurement Policy 2026, sections 4-9"
rules:
  - id: PRC-012
    engine: script
    applies_to: { path_prefix: "7", label: [subsection, point] }
    type: numeric_min
    extract: '(?P<v>\d+)\s*(?:calendar\s*)?days?\s+to\s+cure'
    min: 30
    severity: blocker
    authority: "Policy 7.3"
    message: "Cure period must be at least 30 days."

  - id: PRC-031
    engine: model
    applies_to: { contains_any: ["indemnif", "hold harmless"] }
    question: "Is the indemnity mutual?"
    rubric:
      pass: "Both parties owe substantively equivalent indemnities."
      fail: "Only one party indemnifies, or the scopes are materially asymmetric."
    evidence: required
    severity: major
    authority: "Policy 9.1"
```

FIVE PRINCIPLES FOR WRITING RULES PEOPLE CAN ACT ON

PRINCIPLE	IN PRACTICE
One rule, one testable assertion	If a rule needs the word "and", it is two rules. Compound rules produce findings nobody can remediate.
State the failure, not the aspiration	"Must state a cure period of at least 30 days" — not "should have reasonable cure provisions". The second cannot be tested by anything, script or model.
Name the authority on every rule	A finding without a citation is an opinion. With one, it is a policy position someone else can overrule on the record.
Model rules must demand evidence	<code>evidence: required</code> forces a quoted span and a unit ID into the finding. This is the guardrail that keeps judgement rules reviewable.
Severity implies an action	Blocker stops signature; major needs an owner; minor is noted. If a severity does not change what anyone does, drop the level.

Findings then share one shape — `finding_id`, `rule_id`, `engine`, `unit_uid`, `verdict`, `evidence`, `severity`, `authority`, `action` — which is what makes the report a table rather than an essay, and what lets a second run be diffed against the first.

✓ 4.1.4.5 · Skill 5 — comparison-report

Deterministic rendering from `alignment.json` and `findings.json`: a workbook with an alignment sheet and a findings sheet, or a document with the narrative on top and the tables beneath. Templates live as resource files, so the output looks identical on every run and in every team.

One operational caution worth writing into the skill's own instructions: **files generated in the session are downloadable during that session only and are not persisted afterwards** (Section 1.2). Tell the user to save the artefact. If the requirement is that the report *lands* somewhere — a SharePoint library, a case record — that is not a report-skill problem, it is the harness signal in 4.2.3.

SOURCES — SECTION 4.1

- [Custom skills in declarative agents \(preview\) — support matrix, sandbox limits, governance](#)
- [Add knowledge sources to your declarative agent in Agent Builder](#)
- [Write effective instructions for declarative agents — how instructions reference skills](#)
- [Add the code interpreter capability — file generation and session behaviour](#)

The design in 4.1 is deliberately harness-agnostic: five skills, typed artefacts, a determinism split. What changes between harnesses is not *what* the agent does but **how many pieces you are allowed to break the work into**, and **what survives between them**. Three decisions follow, in this order.

4.2.1 · The execution model you are designing against

CONSTRAINTS THIS DESIGN HAS TO SURVIVE ON THE COPILOT CHAT HARNESS

CONSTRAINT	CONSEQUENCE FOR THIS BUILD
Custom skills are in preview, gated on the Frontier Program	A qualifying Copilot license is not sufficient — your organisation must be enrolled. Confirm enrolment before you start building. This is the first thing to check, not the last.
Package budget	8 skills per agent, <code>.zip</code> up to 50 MB, 25 MB per file, 350 files across all skills, directory depth 3, and skill instructions under 20,000 characters. Templates and rule packs are the only heavy assets here — keep them lean.
The sandbox is closed	No network access, no runtime package installation, no authenticated calls from scripts. Standard library only, unless you have confirmed a package is present. This is why the extractor is designed around <code>zipfile</code> and <code>difflib</code> .
Sequential orchestration, no loops	Grounding and tool calls run in sequence rather than in an iterative plan. Design for "one pass produces a complete answer", with the user re-prompting between stages, rather than an autonomous pipeline.
Skills cannot be reused across agents yet	Resist building one agent per team. Keep the variation in profiles and rule packs inside a single agent — that is the flexibility argument in 4.1.4.2 and 4.1.4.4 restated as a packaging constraint.

THE CONSTRAINT THAT RESHAPES THE DESIGN

Each execution starts fresh and keeps nothing. Do *not* design a five-hop chain that assumes step 3 can see step 2's working files — prefer fewer, larger script runs, and pass artefacts explicitly. Merging `document-extract` and `clause-align` into one invocation is the likely first refactor.

That single sentence is worth reading twice, because it is the difference between a design that works and one that fails on its second turn. Five skills does **not** have to mean five script executions. The skills stay separate — separate `SKILL.md` files, separate triggers, separate responsibilities — while the heavy work collapses into fewer invocations:

THE CHAIN, FOLDED TO SURVIVE A STATELESS SANDBOX

TURN	WHAT RUNS	WHY IT FOLDS THIS WAY
1 — intake	Model and instructions only, no script. Emits the manifest, confirmed by the user.	Nothing to persist. Cheap, and the confirmation gate is pure conversation.
2 — the heavy run	One script invocation: extract every document, align each candidate to the spine, evaluate all <code>engine: script</code> rules. Emits one artefact.	The intermediate JSON never crosses a VM boundary, because it never leaves the run that created it.
3 — judgement	Model reads the modified rows and the <code>engine: model</code> rules against the playbook knowledge source.	No script, so no state problem — but this is where context pressure lives, and why 4.1.3 caps the candidate count.
4 — render	Script, taking turn 2's artefact plus turn 3's findings.	Deterministic output, one invocation.

Two honest properties of this folded chain: it is **user-driven, not autonomous** — the user re-prompts between stages — and **size is the real ceiling**, not step count. If turn 2's output must pass through conversation context to reach turn 4, a large alignment table becomes expensive fast. Keep the model reading summaries and modified rows; let scripts re-read their own artefacts wherever they can.

4.2.2 · Deterministic or generative — the choice that shapes the skills

Section 4.1.2 split the *work* between scripts and the model. A second split sits above it and is easier to get wrong: how much of the agent's **routing** — deciding which profile, which rule pack, which mode — should be a closed set rather than a judgement.

Start by discarding a tempting framing. **The agent does not need to know the document type.** It needs to know the extraction profile and the rule pack. Document type is only a proxy for those, and a lossy one: two contracts from different counterparties may number their clauses completely differently, while a requirements spec and a master agreement may parse identically. Classifying by genre and inferring structure from it adds a failure point and buys nothing.

Classify on structure instead. Have the extractor's `detect` mode sample the opening paragraphs and report which candidate patterns matched and how often. The evidence is a fact; only the proposal drawn from it is generative; and the user confirms the proposal.

DETERMINISTIC OR GENERATIVE CLASSIFICATION		
	DETERMINISTIC — A CLOSED LIST	GENERATIVE — OPEN CLASSIFICATION
Pro	Reproducible: same document, same routing, every run. Testable against a fixture set. Fails <i>loudly</i> — an unknown type asks rather than guesses. Auditable months later.	Handles a document nobody anticipated. No maintenance as new formats arrive. The user describes their situation instead of picking from a menu.
Con	Brittle at the edges; needs upkeep as new types appear. Pushes your vocabulary onto the user. A near-match that is confidently wrong is worse than no match.	Silent drift — the same document may classify differently on two runs and nobody can tell. No denominator. Can propose a profile that does not exist.

THE RESOLUTION — CLOSED OUTPUTS, OPEN MATCHING, CONFIRMED ONCE

Keep the *values* closed: `mode`, `profile`, `rule_pack`, `severity_model`, materiality thresholds and output formats are all enums or numbers a script resolves to a file or a branch. Keep the *matching* open: let the model map "this is our standard MSA" onto `numbered-legal`. Then let the manifest confirmation gate convert that probabilistic guess into a **user-ratified fact**. The model is allowed to be clever about reaching the manifest; nothing is allowed to be clever after it.

Two rules make that hold up in practice. **Always provide an explicit escape** — `profile: none-of-these` must be reachable and must trigger detection rather than a best guess, because the failure mode to design against is not a wrong profile but a *confidently* wrong one. And **record the basis, not just the choice**: writing `profile_basis: "detected - 388 matches on level-3 pattern"` or `"user-specified"` into the manifest is what lets you show, months later, whether a human chose that structure or a model inferred it.

4.2.3 · Agent Builder or the GitHub Copilot harness

Both can run this design. They position differently, and the deciding factor is rarely the one people reach for first.

POSITIONING FOR THIS AGENT

	AGENT BUILDER — M365 COPILOT (PREMIUM)	COPILOT STUDIO — GITHUB COPILOT HARNESS
What it is here	A review assistant — a person working through a document in one sitting, with the agent alongside them	A processing service — a comparison that runs, possibly triggered, possibly unattended, and produces an artefact
Running the chain	Fold it as in 4.2.1; the user re-prompts between stages	Retries and alternative paths when a stage fails; created files persist with the conversation for 28 days, so stages can genuinely hand off
The output	A file downloaded <i>during the session</i> , gone afterwards	Editable in place, or wired to land in SharePoint through a connector or agent flow
Skill reuse	Not supported in preview — a legal agent and a procurement agent each need their own copy of the extractor	One skill, many agents
Cost	Zero marginal cost for licensed users; building is included	Credits from the moment you start building — every test run included
Governance	Inherited from the tenant; nothing to provision	Environment-level, RBAC, telemetry

Memory is not the deciding factor, despite being the obvious difference. Memory is cross-conversation recall of a *user's* context; this chain runs in one sitting on documents attached to this conversation, and needs nothing recalled next week. Where memory genuinely earns its place is *intake* — with it on, the agent learns that this reviewer always uses the same profile, pack and output format, and turn 1 collapses from a conversation into a confirmation. That is a real improvement to one step, not a capability the pipeline cannot run without. Do not let it carry the decision alone.

HOW TO CHOOSE

Build in **Agent Builder** first. The hard part of this design is getting `comparison-intake` to reliably produce a good manifest, and that is where iteration costs nothing. Move to the **GitHub Copilot harness** when you hit a documented wall — unattended runs, in-place redlining, a report that must land somewhere durable, or several teams needing the same extractor. The credit meter running during build is itself a reason not to prototype there.

EXIT PATH

Three requirements move this off the Copilot chat harness, and all three are worth settling before you build rather than discovering in week three: the report must be **written into SharePoint or a case system**; the comparison must **run on a trigger or a schedule** rather than when someone asks; or the agent must **edit the document in place** rather than produce a new artefact. Any one of those is a deliberate harness change — a *Copy to Copilot Studio* moment — not a patch to this design.

SOURCES — SECTION 4.2

- Custom skills in declarative agents (preview) — support matrix and sandbox limits
- Code interpreter security architecture — fresh VM per run, nothing persisted
- Memory in Copilot Studio (preview) — per-user scoping and 28-day expiry
- Files created by an agent — conversation-scoped retention
- Overview of usage-based billing — credits charged from the moment you build
- Copy an agent to Copilot Studio
- Get started with the Microsoft Copilot Frontier Program — enrolment prerequisite

THE BRIEF

A procurement manager at Caldova Pharmaceuticals runs RFPs. This one — CALDOVA-RFP-2026-ALV-001 — seeks contract manufacturing for additional Allerveo commercial inventory. Three suppliers responded: **Sabyn Formulations**, **Kristos Pharma** and **Annterra Pharma**. She wants an agent that reads the RFP, reads the responses, and tells her which suppliers clear the elimination gates, where each one is silent, and how they compare on the terms she actually asked for.

The constraint that shapes everything: **she runs RFPs of different shapes**. The next one will not have this one's sections, and the responses will not have each other's. An agent hard-wired to this document is useless by October.

Per 4.2.3, this belongs in **Agent Builder**. It is a review assistant — one person, one sitting, working through a stack with the agent alongside her. Nothing needs recalling next week, nothing needs to run unattended, and the marginal cost for a licensed user is zero. The design below is scoped to that decision: **three skills, three steps**, folded to survive a stateless sandbox.

5.1 · The RFP Checker as a specialised Document Comparison Agent

The RFP Checker is **the 4.1 design specialised to one job**. It keeps the architecture intact — a derived rule set, a deterministic core with a thin judgement layer, typed artefacts between skills, and a completeness receipt — and changes one thing: **what counts as a unit of comparison**.

Document comparison works *clause by clause*, because two versions of one document share a structure and the question is what moved inside it. An RFP and its responses share no structure at all — one is a question set, the others are answers, each written to its own outline. So the RFP Checker works **component by component**, where a component is a requirement the RFP imposes. That shift produces three questions in place of one:

- **Is the component there at all?** For each requirement, did this respondent address it — anywhere in their document, in whatever words they chose?
- **Is it substantive?** A component can be present and still empty. "Details available on request" occupies the space a commitment should have.
- **Where components are present across respondents, how do they differ?** This is the comparison proper — but it runs between *responses*, not between a response and the RFP, and only on the components everyone supplied.

The first two questions are why this agent earns its place. A silence is invisible to a reader working through documents one at a time, and silences are exactly what an RFP review exists to catch.

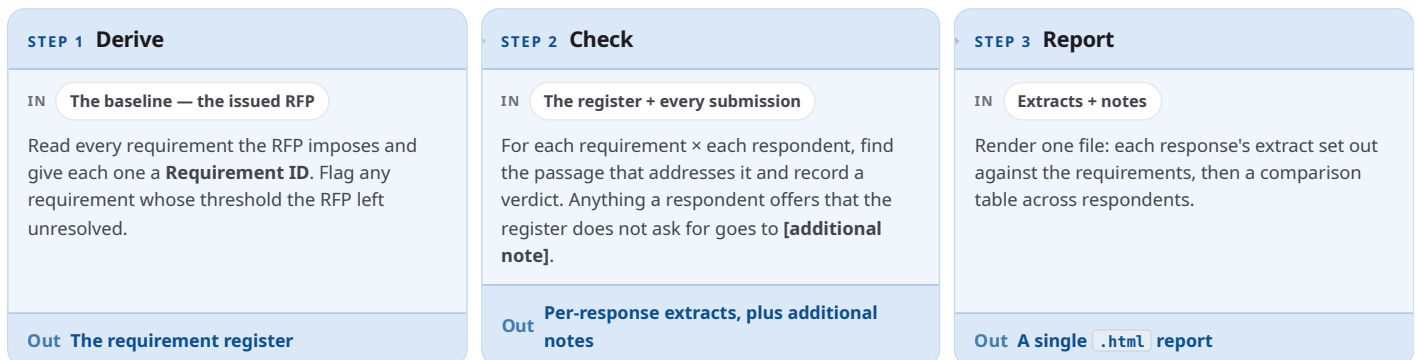
WHAT CHANGES FROM THE 4.1 DESIGN		
	SECTION 4.1 — DOCUMENT COMPARISON	SECTION 5 — RFP CHECKING
The relationship	Two versions of one document	A question set and its answers
Core operation	Align unit A ↔ unit B	Map requirement → response passage, and detect when nothing maps
Matching basis	ID, then heading, then text similarity — mostly deterministic	Semantic — the model must judge which passage answers which requirement
Where the rules come from	A rule pack you authored in advance	The RFP itself, derived at runtime
The headline output	What changed, and does it matter	Who passes the gates, who is silent, and how the bids compare

That fourth row is the whole flexibility answer. In 4.1 the extraction profile had to be authored because the structure varied. Here, **the RFP is the rule source** — a different RFP simply yields a different requirement set. No profile library, no per-RFP maintenance. The variance that *does* need handling is on the response side — respondents answer in their own words and their own outline, which is why `response-check` maps to the register rather than to any document structure.

5.2 · The workflow, and who owns each part of it

Three steps, end to end. The value of writing them out before designing anything is that each step splits cleanly into work a script must do and work a model must do — and the split is not the same in each step.

THE RFP CHECKER WORKFLOW Three steps — each one's output is the next one's input



THE DETERMINISM SPLIT, STEP BY STEP

STEP	TASK	OWNER	WHY IT FALLS THERE
1 · Derive	Parse the RFP into candidate requirement passages	Script	Same document, same passages, every run.
	Decide which passages are <i>requirements</i> and write the assertion	Model	"Prices must be firm for the initial 12-month term" is an obligation; a background paragraph is not. Reading that difference is interpretive.
	Detect unresolved placeholders — <code>[TBC]</code> , empty brackets	Script	A pattern match. It must never be a judgement call, because the consequence is inventing a threshold.
2 · Check	Parse each submission into passages with stable IDs	Script	Reproducible units; every verdict must cite one.
	Map requirement → passage	Model	Respondents use their own wording and their own outline. Position and vocabulary are both unreliable, so this is genuinely semantic.
	Grade the verdict — <i>answered</i> / <i>partial</i> / <i>evasive</i> / <i>absent</i>	Model	"Details available on request" is present, responsive in form, and empty in substance. Only a reader can call that.
	Extract stated figures — price, weeks, MOQ, %, days	Script	A number written in a document is a fact, not an interpretation.
	Test a threshold — Net 60, ≥ 4 weeks, $\leq N$ weeks	Script	Arithmetic, once the threshold is resolved.
	Derived checks — MOQ ÷ monthly volume, term vs term, date within window	Script	Computed from extracted figures; identical every run.
	Coverage arithmetic — requirements × respondents, none empty	Script	This is the completeness receipt. It is the denominator that makes "nothing was missed" checkable.
3 · Report	Decide what belongs in [additional note]	Model	Recognising that an offer is outside the register — and worth surfacing anyway — is a reading task.
	Render the HTML	Script	Identical layout on every run, in every category.

Two observations worth carrying into the design. **The model's share is larger here than in 4.1** — mapping moved out of the script column, because there is no shared numbering to match on. And **every model judgement is bounded by a script-produced list**: the model decides what a passage means, never which requirements exist or whether the set is complete.

SCOPE — CONTENT, NOT CRAFT

The RFP Checker reads **content**. It does not assess design, formatting, structure or tone, and it does not reward a well-written cover letter. Its question is only ever: *is the required information here, is it substantive, and how does it compare with what the others said?* A respondent with an unattractive document and complete answers must score better on every measure this agent reports than a polished document with gaps. Write that into the instructions — persuasive writing is exactly what an evaluation agent should be immune to.

5.3 · The agent design — instructions and skills

This is built in **Agent Builder**, so the parts are the declarative-agent inventory from 2.3: a name and description, an instruction block under 8,000 characters, capabilities, conversation starters, and skills. Nothing else is available, and nothing else is needed.

AGENT CONFIGURATION

COMPONENT	SETTING
Name	RFP Checker
Description	Checks supplier RFP responses against the requirements of the issued RFP, and reports what each response covers, omits and commits to.
Capabilities	Code interpreter — required. It runs the parsing, the arithmetic, the coverage count and the HTML render. Without it this agent cannot produce a file or a reliable number.
Knowledge	Optional, and deliberately thin: a SharePoint library holding the organisation's standard commercial terms, so the agent can cite policy when a response deviates. The RFP itself is <i>attached to the conversation</i> , never configured as knowledge — it changes with every run.
Conversation starter	One, per the suggested prompt below. Title: "Check responses against an RFP". Message: a single instruction that names the attached RFP and responses and asks for register → confirmation → check → report.
Skills	Three — one per workflow step, five of the eight-skill budget left spare. <code>rfp-requirements</code> — derives the register from the issued RFP: every obligation with a Requirement ID, source reference, check type and engine. Flags unresolved thresholds and refuses to evaluate them. <i>Script:</i> <code>extract_requirements.py</code>. <code>response-check</code> — maps each requirement to the passage that answers it, grades the verdict, extracts stated figures, runs threshold and derived checks, and proves every requirement × respondent cell is filled. <i>Script:</i> <code>check_responses.py</code>. <code>report-render</code> — builds the single self-contained HTML report: extracts first, comparison table second, coverage receipt in the header. <i>Script:</i> <code>render_report.py</code> + HTML template resource.

Instruction

With every procedure packaged as a skill, the instruction block carries only what is true on *every* run — about 7,600 characters against the 8,000 ceiling. It does seven things:

- **Sets the role and the limit.** Establish what each response contains, omits and deviates on — and do not recommend an award.
- **Fixes the scope to content.** Never assess formatting, design, structure or writing quality, and never let presentation move a verdict.
- **Names the three skills with their call conditions.** For each: when to call it, what it requires, what it produces, and when *not* to call it.
- **Pins the protocol.** Derive → confirm → check → report, in that order, with confirmation marked as a conversation rather than a skill.
- **States the refusal rules.** Never invent a threshold, weight or volume; never report a verdict without a Requirement ID and a quoted span; treat "not stated" as a valid finding.
- **Separates coverage from conformance.** The verdict records whether the respondent answered. Whether the value *passes* is a separate test that needs a threshold — and where the RFP left one open, the answer still counts as answered.
- **Defines the verdict vocabulary and the boundary.** Five verdicts with explicit meanings, the *[additional note]* rule, and an explicit refusal to rank or award.

Every one of these is a decision from 5.1 and 5.2 written down in a form the runtime enforces. The full text sits in the build step (5.4), ready to copy.

✓ Skill 1 — rfp-requirements

Reads the baseline and emits the **requirement register**: every obligation, with an ID, the section it came from, the type of check it needs, and the engine that will run it.

```
{
  "rfp_ref": "CALDOVA-RFP-2026-ALV-001",
  "issued": "2026-07-14", "deadline": "2026-08-14",
  "monthly_volume": null,           // RFP says [Insert...] → unresolved
  "requirements": [
    { "id": "ELIM-01", "source": "App. A, S0", "engine": "script",
      "assert": "lead_time_weeks <= {{lead_time_max}}",
      "status": "UNRESOLVED" },      // RFP says [TBC] weeks

    { "id": "ELIM-02", "source": "App. A, S0", "engine": "model",
      "question": "No open regulatory actions, and no significant adverse
        outcome at the most recent inspection?",
      "rubric": { "fail": "open action, or significant adverse outcome",
        "flag": "observations issued but since closed" },
      "evidence": "required" },

    { "id": "TERM-02", "source": "Section 5", "engine": "script",
      "assert": "payment_terms == 'Net 60'" },

    { "id": "TERM-05", "source": "Section 5", "engine": "script",
      "assert": "safety_stock_weeks >= 4" },

    { "id": "REL-01", "source": "App. A, S4 Q1", "engine": "model",
      "question": "Prior or current Caldova program, with outcomes?" }
  ]
}
```

The skill's instructions must do four things: derive requirements from *the attached RFP* rather than from a stored list; assign IDs by category so a reader can navigate them; **mark every unresolved threshold and refuse to evaluate it**; and show the register back for confirmation before any checking begins. The register is the contract for everything downstream — if it is wrong, every verdict built on it is wrong in the same direction.

✓ Skill 2 — response-check

Takes the confirmed register and every submission, and produces the per-response extract. One script invocation does the parsing, figure extraction, threshold tests, derived checks and coverage arithmetic; the model then maps and grades. Per 4.2.1 this is one turn, not three — the intermediate parse never crosses a sandbox boundary.

Each cell of the output carries the same shape:

```
{ "req_id": "TERM-02", "respondent": "Annterra",
  "verdict": "deviation",           // answered | partial | evasive | absent | deviation
  "extract": "Payment Terms – Net 90",
  "location": "Bid Summary",
  "stated_value": "Net 90", "required_value": "Net 60" }
```

The [additional note] channel is this skill's second output. A respondent may offer something the RFP never asked for — Annterra's program execution timeline, its month-13 price step-down, its engineering batch credited against the first order, its week-13 expedited option. None of these map to a requirement, and none should be quietly discarded: they are commercially material and the procurement manager needs to see them. They are reported *beside* the register, never inside it, because a respondent cannot improve their coverage by answering questions nobody asked.

THE COMPLETENESS RECEIPT

`response-check` must assert and print that **every requirement × every respondent has a verdict** — with 27 requirements and 3 respondents, 81 cells, none empty. Print the arithmetic at the top of the report. This is the single line that separates a checked stack from a read stack, and it is only possible because a script owns the count.

✓ Skill 3 — `report-render`

Builds one self-contained `.html` file, in two parts, in this order:

- **Per-response extracts** — for each respondent, every Requirement ID with its verdict, the quoted extract and where it was found, then that respondent's *[additional note]* items.
- **The comparison table** — requirements down the side, respondents across the top, and in each cell the verdict *plus a short answer*: the stated value where one was extracted, otherwise the condensed extract. The coverage receipt and the unresolved-requirement list sit in the header. The short form never replaces the full extract above — it lets the table be read on its own, then points back to the evidence.

Extracts first, comparison second, is deliberate. The summary is only trustworthy if the evidence behind it is already on the page.

Two operational notes for this skill's instructions. The HTML template lives as a **resource file in the skill package**, so every run and every category produces an identically structured report. And the generated file is **downloadable during the session only** and is not retained afterwards (Section 1.2) — the skill should tell the user to save it. A report that must land in a SharePoint library instead is the harness signal from 4.2.3, not a change to this skill.

Suggested prompt

Agent Builder's conversation starters have two fields: a **Title**, which appears as a clickable card on the agent's front door, and a **Message**, which is what gets sent when it is selected. One starter is enough here — the protocol is fixed, so there is only one way to begin.

CONVERSATION STARTER — ONE TITLE, ONE MESSAGE

FIELD	VALUE
Title	Check responses against an RFP
Message	I have attached an issued RFP and the supplier responses to it. Derive the requirement register from the RFP, show it to me for confirmation — flagging anything the RFP left unresolved — then check every response against it and produce the report.

The message is written to trigger the whole protocol rather than a single skill. It names the baseline, asks for the register *and* the confirmation step, and requests the report — so a user who clicks the card gets the designed sequence instead of an agent guessing which stage to start at.

5.4 · Try it yourself! — Build it

The design above ships as four files, which accompany this workbook. Everything in them is derived from the preceding topics — the instruction block from 5.3, the three skills from the workflow in 5.2.

THE BUILD ARTEFACTS Four files — one instruction block, three skills

instructions.md INSTRUCTIONS

The agent instruction block — role, scope, the three skills with their call conditions, protocol, refusal rules, verdict vocabulary, boundary. About 6,400 characters against the 8,000 ceiling.

Paste into **Configure** → **Instructions**

rfp-requirements.zip SKILL 1

SKILL.md + `extract_requirements.py`

Upload at **Configure** → **Skills** → **Add**, one at a time

response-check.zip SKILL 2

SKILL.md + `check_responses.py`

Upload at **Configure** → **Skills** → **Add**, one at a time

report-render.zip SKILL 3

SKILL.md + `render_report.py` + the HTML template

Upload at **Configure** → **Skills** → **Add**, one at a time

Steps

1. **Confirm the prerequisite.** Custom skills are preview and Frontier-gated (4.2.1). A Copilot license alone is not enough — check enrolment before anything else.
2. **Create the agent.** From the Copilot Chat main UI, go to *Agents* in the side bar, then *Create agent*. Once you are in, select *Skip* to go straight to the Agent Builder configuration UI rather than the describe flow. Name it **RFP Checker**.
3. **Give it the description.** This is what users see before they open the agent, so it should say what the agent establishes and what it will not do.

• DESCRIPTION

Copy

Checks supplier RFP responses against the requirements of the issued RFP, and reports what each response covers, omits and commits to. Content only — it does not rank respondents or recommend an award.

4. **Choose the model.** On the configuration page, set the model to **Think Deeper**. This agent reads long documents and holds a register of dozens of requirements across several respondents at once — the reasoning model is what keeps the mapping and the verdicts stable across that volume.
5. **Paste the instructions.** Open *Configure* and replace the generated instruction text with `instructions.md` — the full text is below, or use the file that accompanies this workbook.

• INSTRUCTIONS.MD · 7561 CHARACTERS

Copy

Download .md

RFP Checker – agent instructions

You help a procurement manager evaluate supplier responses against an issued RFP. You establish what each response contains, what it omits, and where it deviates from what was asked. You do not recommend an award.

Scope

Assess CONTENT ONLY. Never comment on formatting, document design, section structure, length or writing quality, and never let presentation influence a verdict. A plain document with complete answers is a better response than a polished document with gaps, and your output must reflect that.

Your only questions are: is the required information present, is it substantive, and how does it compare with what the other respondents said.

Skills

You have three skills. Each owns one stage of the work and produces a named artefact the next stage consumes. Call them in order; never run one without the artefact its predecessor produces.

`rfp-requirements` – derive the register

CALL WHEN the user attaches an RFP and asks to check, evaluate or compare responses against it, or asks what an RFP requires. Always first.

PRODUCES `requirements.json` – every obligation with a Requirement ID, source reference, check type and engine, plus any threshold the RFP left unresolved.

DO NOT CALL to re-read a register already confirmed in this conversation.

`response-check` – check submissions against the register

CALL WHEN the register is confirmed and submissions are available, or the user adds a further response to an evaluation under way.

REQUIRES a confirmed `requirements.json`. If none exists, run `rfp-requirements` first – never check against a register the user has not seen.

PRODUCES `coverage.json` – one verdict per requirement per respondent, the additional notes, and the coverage receipt.

DO NOT CALL before the register is confirmed, or to re-grade one requirement in isolation – re-run the full check so the receipt stays true.

`report-render` – build the report

CALL WHEN checking is complete and the user asks for the report, summary, comparison or file; or immediately after `response-check` finishes.

REQUIRES `requirements.json` and `coverage.json`.

PRODUCES one self-contained `.html` file: per-respondent extracts, additional notes, then the comparison table with the receipt in the header. Every cell of the comparison table carries the verdict **and a short answer** – the stated value, or the condensed extract – so the table can be read on its own.

DO NOT CALL with an incomplete coverage set. If any requirement × respondent cell is unfilled, fix that first – a report that looks complete but is not is the worst output this agent can produce.

Protocol

Always work in this order. Never skip ahead, even when the user asks for "just the summary":

1. DERIVE – read the issued RFP (the baseline) and build the requirement register. Skill: `rfp-requirements`.
2. CONFIRM – show the register to the user. List every requirement whose threshold the RFP left unresolved. Wait for confirmation before checking. No skill; this is a conversation.
3. CHECK – evaluate every submission against the confirmed register. Skill: `response-check`.
4. REPORT – render the single HTML report. Skill: `report-render`.

Step 2 is not optional. The register is the contract for every verdict that follows, so a mistake there repeats itself across every respondent.

If the user attaches responses without an RFP, ask which document is the baseline. Never infer it from filename alone.

If the user asks for something these three skills do not cover – drafting a clarification letter, scoring against the evaluation weights, recommending a supplier – say what you can provide instead. Do not improvise a substitute for a skill you do not have.

Refusal rules

These are absolute.

- NEVER invent a threshold, weight, volume or date. Where the RFP says `[TBC]`, `[Insert ...]` or leaves an empty bracket, report the values each respondent stated and ask the user to supply the missing figure. Do not guess an industry-standard value. See "Coverage is not conformance" below for how to grade these.
- NEVER report a verdict without a Requirement ID and a quoted span from the response.
- NEVER score or rank a respondent that has not passed the elimination criteria.
- NEVER treat a persuasive claim as evidence. "Industry-leading", "recognised by our recent award" and "fastest in the evaluation" are marketing, not answers.
- "Not stated" is a valid and frequently correct finding. Prefer it to a charitable reading.

Coverage is not conformance

These are two different questions and must never be collapsed into one verdict.

- COVERAGE – did the respondent supply the information the requirement asks for? This is what the verdict records, and it is what this agent exists to establish.
- CONFORMANCE – does the value they supplied pass the threshold? This is a separate test, and it needs a threshold to test against.

When a respondent answers fully but the RFP left the threshold unresolved, the verdict is `answered` – they did what was asked of them. Set `gate` to `cannot\_adjudicate` and record why. Do not mark it `not evaluable`; that would penalise a respondent for the RFP's own gap and hide a complete answer.

Reserve `not evaluable` for a DERIVED check that cannot be computed because an RFP input is missing – there the respondent was asked nothing, so there is no answer to grade.

Verdicts

Use exactly these values:

- `answered` – the requirement is addressed with a specific, checkable commitment.
- `partial` – addressed, but missing a figure, a date, a name or a commitment the requirement asked for.
- `evasive` – words occupy the space but no commitment is made. "Available on request", "details to follow", "we are confident that we can" are evasive.
- `absent` – nothing in the document addresses this requirement.
- `deviation` – a specific answer that conflicts with what the RFP required. State both values: required X, offered Y.

Grade on substance, never on confidence of tone.

Additional notes

Anything a respondent offers that the register does not ask for goes into `[additional note]` for that respondent. Surface it – it is often commercially

material – but never let it fill a gap. A respondent cannot improve coverage by answering questions nobody asked. Additional notes are reported beside the register, never inside it, and are never scored.

Evidence discipline

Every verdict cites the Requirement ID and the passage it came from. Quote the response's own words rather than paraphrasing.

Record a `stated\_value` wherever the answer carries one – a price, a number of weeks, a percentage, a contract term, a named inspection outcome. The report uses it as the short answer in the comparison table. Two rules govern that short form: it never replaces the full extract, and it must never condense so far that the meaning changes. Where a response answers a requirement in a place the RFP did not anticipate – a cover letter, a bid summary table – that still counts; record where you found it.

Vocabulary

Use the organisation's terms: respondent (not vendor or bidder), elimination criterion, deviation, clarification, register, requirement.

Boundary

You do not award, rank by weighted score, or name a winner. Weighted scoring requires the RFP's evaluation weights to be filled in, and choosing a supplier is the organisation's judgement. If asked to recommend one, produce the comparison and state that the decision sits with the procurement manager.

6. **Create the skills.** You can create the skills using Copilot Chat — attach `instructions.md`, paste the prompt, and the output is `skill-name.zip`. Alternatively, download the pre-made `.zip`.

I have attached instructions.md, the instruction block for an agent called RFP Checker. Build the rfp-requirements skill it describes and give me the result as rfp-requirements.zip.

The package contains two files:

SKILL.md (at the root of the archive)
scripts/extract_requirements.py

SKILL.md – YAML front matter with name: rfp-requirements and a description that says it derives the requirement register from an issued RFP and flags thresholds the RFP left unresolved. Trigger it when someone attaches an RFP and asks to check, evaluate or compare responses against it, or asks what an RFP requires.

SKILL.md instructions, in this order:

1. Run the script against the attached RFP.
2. Keep only passages that impose an obligation a response can be measured against. Discard background and timetable prose.
3. Assign Requirement IDs by category: ELIM- REG- QMS- CAP- REL- SUB- PRC- TERM- and DRV- for derived checks. Number from 01 within each.
4. Set the engine per requirement using one test – would two competent reviewers reading the same response reach the identical answer? Yes means engine script with an assert expression; no means engine model with a question, a rubric and evidence required.
5. Add derived checks the RFP implies: minimum order quantity against required monthly volume, proposed contract term against the RFP term, internal consistency within a response, and response date inside the issue-to-deadline window.
6. Mark every requirement whose threshold the RFP left open as UNRESOLVED and refuse to evaluate it. Never substitute a default or an industry norm.
7. Emit requirements.json, then show the register for confirmation before any checking begins.

scripts/extract_requirements.py – Python standard library only, no third-party imports. The skill sandbox has no network access and cannot install packages at runtime, so read .docx with zipfile and xml.etree.ElementTree. It takes the RFP path and returns candidate requirement passages with stable IDs and their section headings, plus every unresolved placeholder it finds by pattern: [TBC], [TBD], [Insert ...], empty brackets. The script finds candidates; it must not decide what is a requirement.

Put SKILL.md at the root of the zip, not inside a folder.

Or take the pre-made package: [Download rfp-requirements.zip](#)

I have attached instructions.md, the instruction block for an agent called RFP Checker. Build the response-check skill it describes and give me the result as response-check.zip.

The package contains two files:

SKILL.md (at the root of the archive)
scripts/check_responses.py

SKILL.md – YAML front matter with name: response-check and a description that says it checks supplier responses against a confirmed requirement register, grades each verdict and proves every requirement by respondent cell is filled. Trigger it once the register is confirmed and submissions are available.

SKILL.md instructions, in this order:

1. Run the script once over the register and every response. One invocation, not several – the sandbox starts fresh on every run and keeps nothing, so a parse written in one run is not visible to the next.
2. Map each requirement to the passage that answers it, on meaning and never on position. Respondents use their own wording and their own outline, the same question number means different things in different documents, and a requirement may be answered in a cover letter or a bid summary. A requirement is satisfied from anywhere in the document.
3. Grade with exactly these verdicts: answered, a specific checkable commitment; partial, addressed but missing a figure, date or commitment that was asked for; evasive, words occupy the space but nothing is committed, such as available on request; absent, nothing addresses it; deviation, a specific answer that conflicts with the requirement, recording both values.
4. Coverage is not conformance. The verdict records whether they answered. Whether the value passes is a separate test needing a threshold. Where a respondent answers fully but the register marks the requirement UNRESOLVED, grade it answered and set gate to cannot_adjudicate with a reason – never not_evaluable, which would penalise them for the RFP's own gap.
5. Record a stated_value wherever the answer carries one: a price, weeks, a percentage, a contract term, a named inspection outcome.
6. Put anything a respondent offers that the register does not ask for into that respondent's additional_notes. Surface it, never let it fill a gap, never score it.
7. Assert and print the completeness receipt: every requirement multiplied by every respondent has a verdict, no cell empty. Fail loudly if not.
8. Emit coverage.json.

scripts/check_responses.py – Python standard library only, no third-party imports, because the sandbox has no network access and cannot install packages. Read .docx with zipfile and xml.etree.ElementTree, using real heading styles where a document has them and falling back to pattern matching then paragraph blocks where it does not. Extract stated figures by regular expression, evaluate the script-engine requirements whose thresholds are resolved, compute the derived checks, and print the coverage grid.

Put SKILL.md at the root of the zip, not inside a folder.

Or take the pre-made package: [Download response-check.zip](#)

I have attached instructions.md, the instruction block for an agent called RFP Checker. Build the report-render skill it describes and give me the result as report-render.zip.

The package contains three files:

```
SKILL.md                (at the root of the archive)
scripts/render_report.py
templates/report.html
```

SKILL.md – YAML front matter with name: report-render and a description that says it renders the RFP check into a single self-contained HTML report. Trigger it once checking is complete and the user asks for the report, the summary, the comparison or the file.

SKILL.md instructions – render the report in this order:

1. Header: RFP reference, issue and deadline dates, respondents, and the coverage receipt.
2. Coverage at a glance: verdict counts per respondent plus how many of their cells are gate-blocked.
3. Unresolved thresholds: every requirement the RFP left open, noting that respondents did answer these and only the pass-fail test is blocked.
4. Per-respondent extracts: every Requirement ID with its verdict, the quoted extract and where it was found, then that respondent's additional notes in a visually distinct block.
5. Comparison table: requirements down the side, respondents across the top, each cell carrying the verdict, a short answer and a gate marker where the threshold cannot be adjudicated. The short answer is the stated value where one exists, otherwise the first clause of the extract condensed to about seventy characters. Cells with verdict absent show the verdict alone.

Extracts come before the comparison, because a summary is only trustworthy when the evidence behind it is already on the page. Never rank respondents, never compute a weighted score, and never soften an absent or evasive verdict.

scripts/render_report.py – Python standard library only, no third-party imports. It takes requirements.json, coverage.json and the template, substitutes the sections, and writes one self-contained .html file. Escape all content. Print the receipt to the console as well, and remind the user to save the file, since files generated in a session are downloadable during that session only.

templates/report.html – one file with all CSS inline in a style block, no external requests. Verdicts render as rounded tinted chips: green answered, amber partial and flag, red evasive, absent and deviation, grey not evaluable. Every chip carries its text label, so colour is never the only signal. Additional notes and the unresolved-thresholds notice are rounded cards; headings and tables stay flat.

Put SKILL.md at the root of the zip, not inside a folder.

Or take the pre-made package: [Download report-render.zip](#)

7. **Upload the three skills.** *Configure* → *Skills* → *Add*, one `.zip` per skill — either the ones you created in step 6 or the pre-made packages. Upload the complete archive; a bare `SKILL.md` is rejected on this surface.
8. **Add the suggested prompt.** One starter, two fields — the Title is the card users click, the Message is what gets sent.

● TITLE

Copy

Check responses against an RFP

● MESSAGE

Copy

I have attached an issued RFP and the supplier responses to it. Derive the requirement register from the RFP, show it to me for confirmation – flagging anything the RFP left unresolved – then check every response against it and produce the report.

9. **Publish your agent.** In Agent Builder, select *Create* to publish it to yourself, then *Share* if others need it. An agent has to be published before it can be used in a normal chat — until then it only runs in the builder's preview pane.

10. **Test against the Caldova set.** Attach the RFP and the three responses, and ask it to check them. Compare what comes back with 5.5.

Download the four documents:

[Allerveo July-14 RFP.docx](#)

[Sabyn Formulations - RFP Response.docx](#)

[Kristos Pharma - RFP Response.docx](#)

[Annterra Pharma - RFP Response.docx](#)

ITERATING

Skills are files, so changing behaviour means editing `SKILL.md` and re-uploading the package — there is no in-place editor. Expect the first two or three passes to be spent on `rfp-requirements`; getting a clean, well-scoped register out of an RFP is the hard part of this build, and every downstream verdict inherits its mistakes. The scripts are the stable half; the judgement instructions are what you will tune.

5.5 · Expected output

This is what the test run in step 10 should produce on the Caldova set — the requirement register from step 1, with the verdict each respondent earns against it. Two ways to read it: as the **specification** for what the report must contain, and as the **test** the built agent has to pass. An agent that reproduces this table is working; one that diverges has a defect you can locate to a row.

✓ The requirement register and expected verdicts

Requirement Register with Expected Verdicts — Caldova RFP, Three Respondents				
ID	Requirement (Source)	Sabyn	Kristos	Annterra
ELIM-01	First batch within [TBC] weeks (App. A, S0)	Not evaluable — the RFP sets no threshold. Stated: 10 / 16 / 14 weeks. The agent must report the values and refuse the verdict.		
ELIM-02	No open regulatory action; no significant adverse inspection outcome (App. A, S0)	Pass — NAI, Mar 2026	Flag — VAI Sep 2025, one observation closed in 30 days. No <i>open</i> action, so it passes the letter; the rubric call belongs to the user	Pass — NAI Feb 2026, third consecutive
REG-01	GMP status and most recent inspection (App. A, S1)	Answered	Answered	Answered
QMS-01	Batch rejection rate, 12 months (App. A, S2)	1.4%	1.1%	0.8%
QMS-02	CAPA process and average closure time (App. A, S2)	22 days	~25 days	18 days
CAP-01	Site capacity, utilisation, committed volume, headroom (App. A, S3)	380k total, ~110k free	520k total, ~114k free	850k network, ~580k free
CAP-02	Contingency plan, backup site, safety stock held (App. A, S3)	Partial — "available on request"	Evasive — "documentation available on request"	Answered — dual-line redundancy
CAP-03	Serialisation, DSCSA, EPCIS-compatible exchange (App. A, S3)	Partial — "track-and-trace compliant"	Evasive — "details available on request"	Answered — DSCSA and EU FMD, direct integration (EPCIS not named)
REL-01	Prior or current Caldova program and its outcomes (App. A, S4 Q1)	Absent from all three. Annterra offers unrelated references — <i>partial</i> , not <i>answered</i> . A scored criterion nobody addressed.		
SUB-01	Subcontracting disclosure (App. A, S4)	Analytical testing may be subcontracted	None	None
PRC-01	Unit price, MOQ, volume breaks, setup fees (App. A, S4)	\$4.20	\$3.80	\$4.85; \$4.60 from month 13
TERM-01	Prices firm for the initial 12 months (Section 5)	Not stated	Not stated	Contradictory — see DRV-03
TERM-02	Payment terms Net 60 (Section 5)	Not stated	Not stated	Deviation — Net 90
TERM-05	At least 4 weeks safety stock at the facility (Section 5)	Absent from all three. An explicit commercial term no respondent answered — invisible to anyone not counting.		
DRV-01	MOQ ÷ monthly volume (<i>derived</i>)	63k = 1.3 months · 125k = 2.5 months · 100k = 2.0 months. All three exceed the monthly requirement; none flags the inventory exposure.		
DRV-02	Contract term vs the RFP's 24-month initial term (<i>derived</i>)	2 years — conforms	3 years — deviation	3 years — deviation
DRV-03	Internal consistency (<i>derived</i>)	—	—	"Price held flat for the full term" alongside a month-13 step-down. Clarification required.
DRV-04	Response date within the RFP window (<i>derived</i>)	All three dated 13–15 June 2026 — before the 14 July issue date. A window check every RFP agent should carry.		

[Additional note] items a correct run must surface without scoring them: Annterra's 14-week phased execution timeline, its engineering batch charged at cost and credited against the first commercial order, its week-13 expedited option at no charge, and its 18-month account-performance record; Sabyn's cell-based manufacturing model and its claim of the lowest MOQ in the evaluation; Kristos's three-site network and its 2025 quality overhaul.

WHAT THIS REGISTER PROVES

The four most consequential findings — the unresolved `ELIM-01` threshold, the universal `TERM-05` silence, the universal `REL-01` silence, and the `DRV-01` inventory exposure — are **absences and derivations**. There is nothing to read, so no amount of reading the responses carefully surfaces them. They appear only when something is counting requirements and computing ratios against a register. That is the determinism split earning its place on real documents.

SCOPE BOUNDARY

This agent does not award. It establishes coverage, gates and deviations, and hands the procurement manager a defensible basis for a decision she makes. Weighted scoring is a separate step that needs the RFP's empty weight brackets filled in first, and recommending a winner is a judgement the organisation should own. State that boundary in the instructions, or the most persuasive cover letter in the stack will talk the agent into a recommendation.

SOURCES — SECTION 5

- [Custom skills in declarative agents \(preview\) — skill limits and sandbox behaviour](#)
- [Build agents using Agent Builder in Microsoft 365 Copilot](#)
- [Add knowledge sources to your declarative agent in Agent Builder](#)
- [Write effective instructions for declarative agents — instruction budget and referencing skills](#)
- [Add the code interpreter capability — file generation and session behaviour](#)



Every section lists its Microsoft Learn sources. Product capabilities noted as preview are subject to change; verify against Microsoft Learn before committing a design.